

**МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ**  
**ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА**  
**ФАКУЛЬТЕТ УПРАВЛІННЯ ФІНАНСАМИ ТА БІЗНЕСУ**

**Кафедра цифрової економіки та бізнес-аналітики**

**КУРСОВА РОБОТА**

з моделювання та автоматизації бізнес-процесів  
на тему:

Інформаційна система автосалону

**спеціальність: 051 “Економіка”**

(код та найменування спеціальності)

**спеціалізація: “Інформаційні технології в бізнесі”**

(найменування спеціалізації)

**освітній ступінь: Бакалавр**

(бакалавр/магістр)

**Науковий керівник:**

к.е.н., доцент Ярема О.Р

(науковий ступінь, посада, прізвище, ініціали)

“ ” 20\_\_ р.  
(підпис)

**Виконавець:**

студент групи УФЕ-31С

Чикель Максим

(прізвище, ініціали)

“ ” 20\_\_ р.  
(підпис)

**Загальна кількість балів** \_\_\_\_\_

\_\_\_\_\_  
(підпис, ППІ члена комісії)

\_\_\_\_\_  
(підпис, ППІ члена комісії)

\_\_\_\_\_  
(підпис, ППІ члена комісії)

**ЛЬВІВ 2026**

# ЗМІСТ

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| ВСТУП.....                                                                              | 3  |
| РОЗДІЛ 1. АНАЛІЗ ВИМОГ ДЛЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ РОБОТИ З<br>КЛІЄНТАМИ АВТОСАЛОНУ ..... | 6  |
| 1.1. Постановка завдання.....                                                           | 6  |
| 1.2. Розробка моделі варіантів використання веб-сайту .....                             | 9  |
| 1.3. Аналіз засобів реалізації (техніко-економічне обґрунтування вибору) ...            | 12 |
| РОЗДІЛ 2. РОЗРОБКА БАЗИ ДАНИХ ІНФОРМАЦІЙНОЇ СИСТЕМИ .....                               | 15 |
| 2.1. Опис моделі даних .....                                                            | 15 |
| 2.2. Нормалізація відношень .....                                                       | 19 |
| 2.3. Визначення типів даних .....                                                       | 20 |
| 2.4. Обмеження цілісності даних .....                                                   | 23 |
| 2.5. Реалізація SQL-скрипту.....                                                        | 25 |
| РОЗДІЛ 3. РОЗРОБКА ВЕБ-ДОДАТКУ ДЛЯ АВТОСАЛОНУ .....                                     | 31 |
| 3.1. Структура веб-сайту .....                                                          | 31 |
| 3.2. Макети сторінок веб-сайту.....                                                     | 33 |
| 3.3. Програмування серверної частини .....                                              | 38 |
| 3.4. Програмування клієнтської частини .....                                            | 45 |
| 3.5. Розміщення веб-сайту на локальному середовищі .....                                | 49 |
| ВИСНОВКИ.....                                                                           | 51 |
| СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ .....                                                        | 53 |

## **ВСТУП**

У сучасних умовах динамічного розвитку автомобільного ринку України та зростання попиту на автомобілі, пригнані з-за кордону, особливої актуальності набуває питання автоматизації бізнес-процесів автосалонів, що спеціалізуються на цьому виді діяльності. Автосалон, який пропонує клієнтам послуги з підбору, купівлі та доставки автомобілів із США та країн Європейського Союзу, оперує великими обсягами інформації: каталогом транспортних засобів, заявками клієнтів, договорами, статусами логістики та фінансовою аналітикою. Ефективна обробка цих даних неможлива без використання сучасних інформаційних технологій.

Впровадження інформаційної системи на базі веб-додатку дозволяє значно підвищити рівень обслуговування клієнтів, скоротити час на обробку заявок, забезпечити прозорість процесу пригону автомобіля від моменту купівлі до передачі покупцеві, а також надати керівництву автосалону зручні інструменти для прийняття обґрунтованих управлінських рішень. Окрім цього, наявність повноцінного клієнтського інтерфейсу з можливістю онлайн-подання заявок, відстеження статусу доставки та залишення відгуків формує конкурентну перевагу автосалону на ринку послуг із пригону автомобілів.

Все це обумовлює актуальність обраної теми курсової роботи та необхідність розробки спеціалізованої інформаційної системи для автосалону, яка враховує специфіку бізнесу, пов'язаного з пригоном автомобілів з-за кордону.

**Метою курсової роботи** є проектування та розробка інформаційної системи для роботи з клієнтами автосалону, що спеціалізується на пригоні автомобілів з-за кордону, із застосуванням сучасних веб-технологій та реляційної бази даних.

**Об'єктом дослідження** є процеси прийому, обробки та обслуговування клієнтів автосалону, що спеціалізується на пригоні автомобілів з-за кордону.

**Предметом дослідження** є теоретичні та практичні засади розробки інформаційної системи з використанням мови SQL (СКБД MySQL), мов розмітки HTML та CSS, а також мови програмування JavaScript на платформі Node.js.

Відповідно до поставленої мети у курсовій роботі сформульовано та вирішено такі основні завдання:

- дослідити предметну область та визначити основні бізнес-процеси автосалону, що займається пригоном автомобілів;
- розробити модель варіантів використання та обґрунтувати вибір засобів реалізації інформаційної системи;
- спроектувати реляційну базу даних з урахуванням вимог нормалізації відношень та забезпечення цілісності даних;

-реалізувати інформаційну систему у вигляді клієнт-серверного веб-додатку та провести її тестування у локальному віртуальному середовищі.

**Практичне значення отриманих результатів.** Розроблена інформаційна система дозволить автоматизувати ключові бізнес-процеси автосалону, мінімізувати ризик помилок при ручному веденні обліку, забезпечити прозорість процесу пригону автомобіля для клієнта та підвищити загальну ефективність роботи підприємства. Запропоновані рішення можуть бути впроваджені у діяльність реально функціонуючих автосалонів та адаптовані під їхні специфічні вимоги.

**Використане програмне забезпечення.** У процесі розробки використано такі програмні засоби: середовище розробки Visual Studio Code (з підключенням до віддаленого сервера через SSH); система керування базою даних MySQL разом із графічним інструментом MySQL Workbench для проектування EER-діаграми та написання SQL-скриптів; платформа Node.js із фреймворком Express.js для побудови серверної частини; стандартні мови веб-розробки HTML5, CSS3 та JavaScript для клієнтської частини; додаткові npm-бібліотеки mysql2, bcrypt, jsonwebtoken та cors. Розгортання здійснено на віртуальній машині VMware під управлінням операційної системи Ubuntu Server.

**Структура курсової роботи.** Курсова робота складається зі вступу, трьох розділів, висновків, списку використаних джерел і додатків. У першому розділі «Аналіз вимог для інформаційної системи» виконано постановку завдання, розроблено модель варіантів використання та обґрунтовано вибір засобів реалізації. У другому розділі «Розробка бази даних інформаційної системи» наведено опис моделі даних, проведено нормалізацію відношень, визначено типи даних та обмеження цілісності, реалізовано SQL-скрипт. У третьому розділі «Розробка веб-додатку для автосалону» описано структуру веб-сайту, його макети, програмування серверної та клієнтської частин, а також розміщення сайту на локальному віртуальному середовищі.

# **РОЗДІЛ 1. АНАЛІЗ ВИМОГ ДЛЯ ІНФОРМАЦІЙНОЇ СИСТЕМИ РОБОТИ З КЛІЄНТАМИ АВТОСАЛОНУ**

## **1.1. Постановка завдання**

Сучасний автомобільний ринок України характеризується стрімким зростанням попиту на автомобілі іноземного виробництва, що пригнані з-за кордону, переважно зі США та країн Європейського Союзу. Така тенденція пояснюється тим, що транспортні засоби, які постачаються за схемою індивідуального пригону, мають кращу комплектацію, нижчу вартість порівняно з аналогами на внутрішньому ринку та більший вибір моделей. Однак специфіка цього бізнесу вимагає чіткої організації роботи з клієнтами, ведення обліку транспортних засобів, оперативного оформлення замовлень і прозорого

відстеження процесу доставки автомобіля від моменту купівлі на закордонному аукціоні до передачі його кінцевому покупцеві.

Метою розробки інформаційної системи для роботи з клієнтами автосалону «GlobalDrive Auto» є автоматизація основних бізнес-процесів підприємства, що спеціалізується на пригоні автомобілів з-за кордону. Розроблена система повинна забезпечити ефективне керування каталогом автомобілів, обробку клієнтських заявок на пригін чи консультацію, оформлення замовлень з фіксацією договірних умов, відстеження логістики доставки в режимі реального часу, а також ведення внутрішньої бази клієнтів та персоналу.

Інформаційна система реалізується у вигляді клієнт-серверного веб-додатку, який доступний через будь-який сучасний браузер та не вимагає від користувачів встановлення додаткового програмного забезпечення. Такий підхід значно спрощує впровадження системи у щоденну роботу автосалону та забезпечує можливість віддаленого доступу як для клієнтів, так і для працівників підприємства.

Розробка веб-сайту автосалону передбачає створення кількох змістовних розділів, які повинні задовольняти інформаційні потреби різних категорій користувачів. На головній сторінці потенційний клієнт ознайомлюється з перевагами співпраці з автосалоном, переглядає основні етапи процесу пригону та отримує загальне уявлення про компанію. У розділі «Каталог авто» представлена повна добірка автомобілів, що наразі доступні до продажу або вже знаходяться в дорозі до України, з можливістю фільтрації за маркою, моделлю, роком випуску, типом палива, коробкою передач та ціновим діапазоном. Розділ «Наші послуги» інформує клієнтів про спектр пропонованих сервісів -від підбору авто на аукціоні до митного оформлення та страхування. На сторінці «Про нас» розкривається інформація про автосалон, його місію та переваги, а у розділі «Контакти» подаються адреса офісу, телефони та форма зворотного зв'язку.

Для зареєстрованих користувачів системи доступний особистий кабінет, у якому клієнт може переглянути власні заявки, відстежити поточний статус

доставки замовленого автомобіля, побачити деталі укладеного договору та залишити відгук про роботу компанії. Подача нової заявки на пригін чи консультацію є однією з ключових функцій клієнтського інтерфейсу, оскільки саме з неї починається ланцюг взаємодії між клієнтом та автосалоном.

Адміністративна частина системи орієнтована на двох категорій працівників -менеджерів та адміністраторів. Менеджер відповідає за оперативну роботу з клієнтами: обробляє вхідні заявки, призначає їм статуси («новий», «в обробці», «виконано», «скасовано»), оформлює замовлення (договори) з фіксацією дати, загальної суми та способу оплати, а також у режимі реального часу оновлює статус логістики кожного автомобіля, що знаходиться в дорозі. Адміністратор має розширені повноваження: окрім функцій менеджера, йому доступне керування каталогом автомобілів (додавання, редагування та видалення записів), керування персоналом та повний доступ до клієнтської бази.

База даних, яка лежить в основі інформаційної системи, забезпечує надійне зберігання та обробку всієї необхідної інформації. Серед основних технічних можливостей, які повинна реалізовувати база даних, можна виділити такі:

- зберігання повної інформації про автомобілі, що знаходяться в каталозі: VIN-код, марка, модель, рік випуску, об'єм двигуна, тип палива, пробіг, колір, тип трансмісії та приводу, країна походження, ціни купівлі та продажу, поточний статус і фотографія;
- ведення реєстру клієнтів автосалону з можливістю розрізняти фізичних і юридичних осіб, зберігання їхніх контактних даних та хешу паролю для авторизації;
- облік заявок клієнтів із зазначенням типу («авто в наявності», «пригін під замовлення», «консультація») і збереженням повного контексту запиту у форматі JSON;
- оформлення замовлень з прив'язкою до конкретного клієнта, автомобіля, заявки та менеджера, що оформив договір, а також з фіксацією дати, загальної суми, статусу оплати та поточного статусу виконання;



- ведення журналу подій логістики кожного замовлення: поточне розташування авто, статус доставки, дата події, очікувана дата прибуття;
- зберігання інформації про персонал автосалону із розрізненням посад («адміністратор» / «менеджер») та контролем прав доступу;
- реалізація багаторівневої системи авторизації з розмежуванням прав між трьома категоріями користувачів (клієнт, менеджер, адміністратор) та контролем доступу до конкретних маршрутів API на серверній стороні;
- безпечне зберігання облікових даних клієнтів і працівників: хешування паролів за криптографічно стійким алгоритмом bcrypt та автентифікація запитів через JSON Web Tokens (JWT) з обмеженим терміном дії;
- захист персональних даних клієнтів: кожен авторизований клієнт має доступ лише до власних заявок, замовлень та інформації про доставку, тоді як працівники бачать дані тільки у межах своїх посадових повноважень;
- реалізація аналітичних запитів для керівництва: кількість заявок за період, ефективність роботи кожного менеджера, найпопулярніші марки автомобілів, обсяг продажів у грошовому виразі тощо.

Запропонована інформаційна система дозволить автоматизувати рутинні операції, мінімізувати ризик помилок, пов'язаних із ручним веденням обліку, а також надати керівництву автосалону зручні інструменти для прийняття обґрунтованих управлінських рішень. Окрім цього, наявність зрозумілого та зручного клієнтського інтерфейсу сприятиме підвищенню рівня довіри клієнтів і зростанню конкурентоспроможності підприємства на ринку послуг із пригону автомобілів.

## **1.2. Розробка моделі варіантів використання веб-сайту**

Розроблювана інформаційна система передбачає три рівні доступу до її функціональних можливостей: клієнт, менеджер та адміністратор. Кожен із цих рівнів характеризується власним набором прав та доступних операцій, що

дозволяє чітко розмежувати зони відповідальності користувачів і захистити чутливу інформацію від несанкціонованого доступу.

Клієнт, як основний зовнішній актор системи, має можливість виконувати такі дії: переглядати каталог автомобілів, користуватися пошуком та фільтрацією за бажаними параметрами, подавати заявку на пригін авто або консультацію щодо вже наявного у каталозі автомобіля, відстежувати поточний статус доставки свого замовлення та залишати відгук про роботу автосалону. Подача заявки, перегляд особистого кабінету, відстеження доставки та залишення відгуку доступні лише після успішної авторизації або реєстрації клієнта у системі, що відображено на діаграмі через відносини типу include.

Менеджер є внутрішнім актором системи, який відповідає за оперативну роботу з клієнтами та обробку їхніх запитів. До його основних функціональних можливостей належать: обробка вхідних заявок (зміна статусу заявки, прив'язка її до конкретного автомобіля з каталогу, переведення у статус «опрацьована»), оформлення замовлення (договору) на основі обробленої заявки із зазначенням загальної суми, способу оплати та поточного статусу виконання, а також зміна статусу логістики та доставки замовленого автомобіля у режимі реального часу. Доступ менеджера до перерахованих функцій також забезпечується через авторизацію у системі.

Адміністратор має найширші повноваження серед усіх акторів системи. Окрім виконання всіх функцій менеджера, йому доступні такі дії: керування каталогом автомобілів (додавання нових записів, редагування існуючих та видалення тих, що втратили актуальність), керування персоналом (додавання нових працівників, зміна їхніх посад, видалення облікових записів) та керування клієнтською базою (перегляд і редагування інформації про клієнтів, видалення облікових записів за потреби).

Для кращого розуміння функціональних взаємодій між акторами та системою було побудовано діаграму варіантів використання (рис. 1.1). Діаграма дозволяє візуалізувати всі основні сценарії використання системи різними категоріями користувачів та виявити зв'язки між варіантами використання, такі як включення (include) і розширення (extend). Наявність відносин типу include відображає обов'язковий зв'язок: для виконання певної дії користувач повинен попередньо виконати іншу (наприклад, для відстеження доставки необхідно бути авторизованим). Відносини типу extend, навпаки, показують можливе, але не обов'язкове розширення базової поведінки (наприклад, перегляд каталогу може бути розширений функцією пошуку та фільтрації).

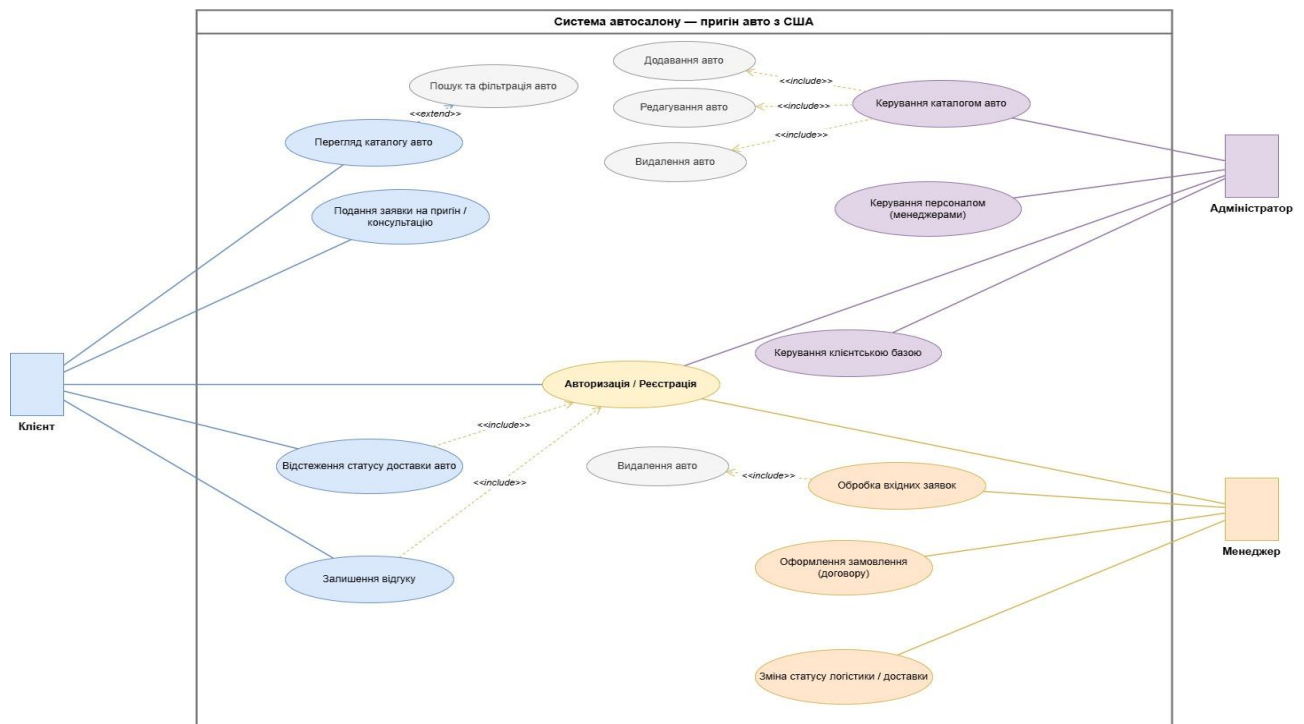


Рис. 1.1. Use Case діаграма інформаційної системи автосалону

*Джерело: Розроблено автором*

Побудована модель варіантів використання слугує основою для подальшого проектування інформаційної системи. Виявлені у процесі моделювання сутності, такі як «Клієнт», «Заявка», «Замовлення», «Автомобіль», «Працівник» та «Доставка», стануть базовими сутностями реляційної бази даних, що буде розроблена у наступному розділі. Інформаційні потоки, які

відображені на діаграмі, будуть реалізовані у вигляді відповідних REST API-маршрутів серверної частини веб-додатку.

### **1.3. Аналіз засобів реалізації (техніко-економічне обґрунтування вибору)**

Розробка повнофункціональної інформаційної системи для автосалону є складним процесом, який вимагає виваженого вибору технологічного стеку. Помилка на етапі вибору засобів реалізації може призвести до значного збільшення трудомісткості розробки, складнощів з подальшим супроводом програмного продукту або неможливості розширення функціональності. Тому перед початком практичної реалізації було проведено порівняльний аналіз сучасних веб-технологій і обрано стек, який оптимально відповідає поставленим завданням.

Сучасний веб-додаток умовно ділиться на дві взаємопов'язані частини: клієнтську (frontend), що виконується у браузері користувача, та серверну (backend), яка обробляє запити клієнта, взаємодіє з базою даних і повертає результат. Окрім цього, окремим компонентом виступає система керування базою даних (СКБД), що відповідає за збереження та надійну обробку даних.

Для розробки клієнтської частини обрано класичний стек технологій **HTML5, CSS3 та JavaScript**. HTML5 використовується як мова розмітки сторінок і забезпечує семантичну структуру документа: заголовки, навігацію, основний контент, форми та інші структурні елементи. CSS3 застосовується для оформлення зовнішнього вигляду сторінок: визначення кольорової гами, шрифтів, відступів, адаптивних правил для коректного відображення на мобільних пристроях. JavaScript додає сторінкам інтерактивність -динамічну фільтрацію каталогу, валідацію форм, асинхронні запити до сервера через Fetch API, обробку відповідей сервера та оновлення інтерфейсу без перезавантаження сторінки. Перевагою такого підходу є відсутність залежності від важких фреймворків (React, Vue, Angular), що зменшує об'єм коду та підвищує швидкість завантаження сторінок. [4, 5]

Для серверної частини обрано платформу **Node.js** з **фреймворком Express.js**. Node.js є середовищем виконання JavaScript на сервері, побудованим на високопродуктивному рушії V8 від компанії Google. Основними перевагами Node.js, які стали вирішальними при виборі цієї платформи, є: висока швидкодія завдяки асинхронній моделі обробки запитів (event-driven, non-blocking I/O); єдина мова програмування для клієнта і сервера, що спрощує розробку і знижує ймовірність помилок при передачі даних; великий екосистемний репозиторій npm з готовими модулями для розв'язання типових завдань; активна спільнота розробників та широка база документації. Фреймворк Express.js надає мінімалістичний, але потужний інструментарій для побудови REST API: маршрутизацію запитів, обробку middleware, інтеграцію з шаблонізаторами та сторонніми модулями. [6, 9, 10]

У ролі системи керування базою даних обрано **MySQL** -одну з найпоширеніших реляційних СКБД у світі. MySQL є вільним програмним забезпеченням з відкритим кодом, що поширюється за ліцензією GPL, забезпечує високу швидкодію при роботі з великими обсягами даних, підтримує стандарт SQL у повному обсязі, має зрозумілий синтаксис та широкі можливості для адміністрування. Для проектування структури бази даних і написання SQL-скриптів використовується середовище MySQL Workbench, що надає графічні засоби для побудови EER-діаграм (Enhanced Entity-Relationship), генерації SQL-коду на основі діаграми та зворотного інжинірингу існуючих баз даних. [3, 11]

Для взаємодії серверної частини з базою даних застосовано бібліотеку **mysql2**, яка надає сучасний асинхронний інтерфейс на основі промісів (Promises) для виконання SQL-запитів з Node.js-додатка. На відміну від класичної бібліотеки **mysql**, **mysql2** має кращу продуктивність, підтримує підготовлені запити (prepared statements), що захищає від SQL-ін'єкцій, і має вбудовану підтримку пулу з'єднань для оптимального використання ресурсів сервера.

Окрім основного технологічного стеку, у проекті використано низку допоміжних бібліотек: **bcrypt** для криптографічно стійкого хешування паролів з автоматичною генерацією символів, що забезпечує надійний захист облікових

даних; `jsonwebtoken` для генерації та перевірки JWT-токенів, які реалізують механізм автентифікації без зберігання сесій на сервері; `cors` для налаштування політики обміну ресурсами між різними джерелами; `multer` для обробки завантаження файлів (зокрема, фотографій автомобілів). [6]

Особливу увагу при розробці інформаційної системи приділено питанням інформаційної безпеки, оскільки система оперує персональними даними клієнтів та конфіденційною інформацією про комерційні операції автосалону. Для забезпечення необхідного рівня захисту реалізовано комплекс заходів. По-перше, паролі користувачів зберігаються у базі даних виключно у вигляді криптографічних хешів, обчислених за алгоритмом `bcrypt` з використанням механізму «солі» (`salt`), що робить практично неможливим відновлення вихідних паролів навіть у випадку несанкціонованого доступу до бази. По-друге, автентифікація користувачів реалізована через підписані JSON Web Tokens (JWT) з обмеженим терміном дії, що дозволяє відмовитися від збереження сесій на сервері та забезпечує масштабованість додатку. По-третє, на серверній стороні впроваджено систему рольових перевірок через `middleware`-функції `Express.js`: для кожного маршруту API визначено, які саме категорії користувачів (клієнт, менеджер, адміністратор) мають право доступу до нього. По-четверте, секретний ключ підпису JWT-токенів та облікові дані для підключення до бази даних винесено в окремий конфігураційний файл `.env`, який не потрапляє до системи контролю версій завдяки відповідним правилам у файлі `.gitignore`. Така багаторівнева архітектура безпеки відповідає сучасним практикам розробки веб-додатків і дозволяє надійно захистити інформаційну систему від типових векторів атак. [12]

Для розгортання серверної частини обрано локальне віртуальне середовище на базі гіпервізора `VMware` з встановленою операційною системою `Ubuntu Server`. Такий підхід дозволяє повністю емулювати умови продакшн-сервера: налаштовувати мережеві параметри, керувати службами, тестувати додаток у контрольованому середовищі. Розробка коду здійснюється у середовищі `Visual Studio Code` з підключенням до віддаленого сервера через

протокол SSH, що забезпечує зручність редагування коду безпосередньо на цільовій машині.

У ролі систем контролю версій та середовищ розробки додатково використано: Git -для відстеження змін у коді проекту, draw.io -для побудови Use Case діаграми, MySQL Workbench -для проектування та керування базою даних.

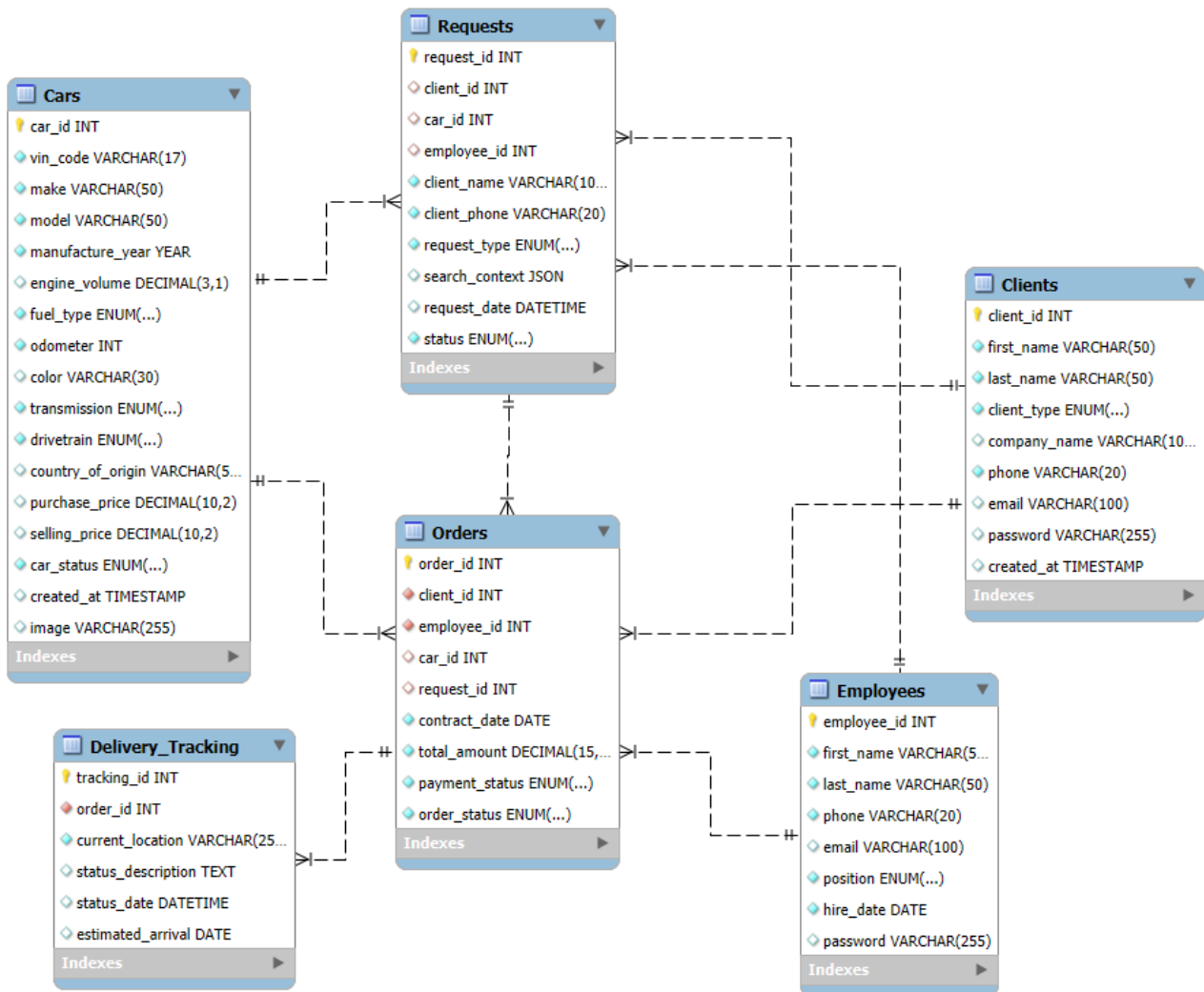
Обраний технологічний стек у поєднанні з продуманою архітектурою додатку забезпечує побудову повнофункціональної, масштабованої та підтримуваної інформаційної системи, що відповідає бізнес-вимогам автосалону «GlobalDrive Auto». Кожен компонент стеку обрано з урахуванням специфіки предметної області: реляційна СКБД MySQL добре підходить для зберігання структурованих даних про автомобілі, клієнтів та замовлення; платформа Node.js забезпечує високу пропускну здатність при обробці одночасних запитів; класичний стек HTML/CSS/JavaScript на клієнтській частині гарантує сумісність із будь-яким сучасним браузером. Окремо варто відзначити, що всі обрані технології поширюються за вільними ліцензіями з відкритим кодом, що повністю усуває витрати на ліцензійне програмне забезпечення при впровадженні системи в реальну діяльність автосалону.

## **РОЗДІЛ 2. РОЗРОБКА БАЗИ ДАНИХ ІНФОРМАЦІЙНОЇ СИСТЕМИ**

### **2.1. Опис моделі даних**

Модель даних інформаційної системи автосалону «GlobalDrive Auto» визначає структуру збереження інформації, взаємозв'язки між основними сутностями предметної області та забезпечує цілісність даних на всіх етапах

роботи з ними. В основі моделі лежить розширена діаграма сутностей і зв'язків (Enhanced Entity-Relationship Diagram, EER-діаграма), яка наочно відображає всі таблиці бази даних, атрибути кожної таблиці та зв'язки між ними (рис. 2.1).



**Рис. 2.1. EER-діаграма бази даних автосалону**

*Джерело: Розроблено автором*

Розроблена база даних складається з шести взаємопов'язаних таблиць, кожна з яких відповідає одній сутності предметної області: Cars (автомобілі), Clients (клієнти), Employees (працівники), Requests (заявки), Orders (замовлення) та Delivery\_Tracking (відстеження доставки). Розглянемо детально призначення та склад атрибутів кожної з таблиць. [1, 2]

Таблиця **Cars** зберігає інформацію про всі автомобілі, які проходять через автосалон - як ті, що вже знаходяться у наявності, так і ті, що в дорозі або вже



продані. Основними атрибутами таблиці є: унікальний ідентифікатор `car_id` (первинний ключ), VIN-код як унікальний номер кузова автомобіля, марка (`make`), модель (`model`), рік випуску (`manufacture_year`), об'єм двигуна у літрах (`engine_volume`), тип палива (`fuel_type` - бензин, дизель, гібрид, електро), пробіг (`odometer`), колір (`color`), тип трансмісії (`transmission` - «автомат», «механіка»), тип приводу (`drivetrain` - передній, задній, повний), країна походження (`country_of_origin`), ціна купівлі на аукціоні (`purchase_price`), ціна продажу (`selling_price`), статус автомобіля (`car_status` - «заброньовано», «на аукціоні», «в дорозі», «в ремонті», «в наявності», «продано»), дата створення запису (`created_at`) та шлях до файлу зображення (`image`).

Таблиця **Clients** містить дані про зареєстрованих клієнтів автосалону. Поля таблиці: `client_id` (первинний ключ), ім'я та прізвище (`first_name`, `last_name`), тип клієнта (`client_type` - «фізична особа», «юридична особа», «vip»), назва компанії (`company_name`, заповнюється лише для юридичних осіб), номер телефону (`phone`) з обмеженням унікальності, електронна адреса (`email`) з обмеженням унікальності, хеш пароля (`password`) та часова мітка створення запису (`created_at`). Поле `phone` використовується як логін при авторизації клієнта у системі, оскільки телефон є більш універсальним та запам'ятовуваним ідентифікатором, ніж `email` - особливо для випадків, коли клієнт залишає заявку як незареєстрований відвідувач і вже після цього створює обліковий запис із тим самим номером..

Таблиця **Employees** зберігає інформацію про працівників автосалону. Її атрибути: `employee_id` (первинний ключ), ім'я та прізвище (`first_name`, `last_name`), номер телефону (`phone`) з обмеженням унікальності, електронна адреса (`email`) з обмеженням унікальності, посада (`position` - «Адміністратор», «Менеджер авто в наявності», «Менеджер з пригону»), дата прийому на роботу (`hire_date`) та хеш пароля (`password`). У базі даних розрізняються три типи працівників: адміністратор з найширшими повноваженнями та два спеціалізованих менеджери - один обробляє заявки на пригін авто з-за кордону,

інший працює з клієнтами, що цікавляться вже наявними у каталозі автомобілями.

Таблиця **Requests** акумулює всі заявки, що подають клієнти через веб-сайт автосалону. Поля таблиці: `request_id` (первинний ключ), `client_id` (зовнішній ключ на таблицю `Clients`), `car_id` (зовнішній ключ на конкретний автомобіль, якщо заявка стосується наявного в каталозі), `employee_id` (зовнішній ключ на менеджера, що обробляє заявку), ім'я та контактний телефон, вказані у заявці (`client_name`, `client_phone` -можуть відрізнитися від основних, якщо заявку подає не зареєстрований користувач), тип заявки (`request_type` - «авто в наявності», «пригін під замовлення», «консультація»), контекст пошуку у форматі JSON (`search_context`, де зберігаються бажані параметри авто: марка, модель, рік, бюджет тощо), дата подання заявки (`request_date`) та її поточний статус (`status` - «новий», «в обробці», «виконано», «скасовано»).

Таблиця **Orders** фіксує укладені договори (замовлення) між автосалоном і клієнтом. Структура таблиці: `order_id` (первинний ключ), `client_id`, `employee_id`, `car_id`, `request_id` (всі -зовнішні ключі), дата укладення договору (`contract_date`), загальна сума замовлення (`total_amount`), статус оплати (`payment_status` - «очікує оплати», «частково сплачено», «оплачено») та статус виконання замовлення (`order_status` - «новий», «в процесі», «завершено», «скасовано»).

Таблиця **Delivery\_Tracking** призначена для ведення журналу подій логістики кожного замовлення. У ній зберігаються: `tracking_id` (первинний ключ), `order_id` (зовнішній ключ на таблицю `Orders`), поточне розташування автомобіля (`current_location`), опис статусу доставки (`status_description`), дата події (`status_date`) та очікувана дата прибуття (`estimated_arrival`). Одне замовлення може мати кілька записів у цій таблиці, що утворюють хронологічний журнал переміщення автомобіля від моменту купівлі до доставки клієнту.

Основні зв'язки між таблицями реалізують типові бізнес-відношення предметної області. Один клієнт може подати декілька заявок (зв'язок «один-до-багатьох» між `Clients` і `Requests`) та оформити декілька замовлень (`Clients` →

Orders). Один автомобіль може бути предметом декількох заявок від різних клієнтів, але лише одного фінального замовлення (Cars → Requests, Cars → Orders). Один менеджер обробляє багато заявок і оформлює багато замовлень (Employees → Requests, Employees → Orders). Кожне замовлення має зв'язок із заявкою, на основі якої воно сформоване (Requests → Orders), і може мати декілька записів про логістику (Orders → Delivery\_Tracking).

## **2.2. Нормалізація відношень**

Нормалізація реляційних відношень -це процес послідовного перетворення схеми бази даних з метою усунення надмірності даних, аномалій оновлення, вставки та видалення, а також забезпечення логічної цілісності інформації. У процесі проектування бази даних автосалону було проведено перевірку розробленої схеми на відповідність першим трьом нормальним формам. [1, 2]

**Перша нормальна форма (1НФ).** Відношення знаходиться у першій нормальній формі, якщо всі його атрибути є атомарними, тобто не містять списків значень, повторюваних груп або структурованих складених типів даних. У розробленій базі даних усі атрибути таблиць задовольняють цій вимозі: кожне поле зберігає одне неподільне значення відповідного типу. Наприклад, у таблиці Clients контактний телефон та електронна пошта представлені окремими полями (phone, email), а не одним полем з кількома значеннями. Виняток становить поле search\_context таблиці Requests, що має тип JSON, проте у даному контексті воно використовується саме для збереження неструктурованого допоміжного контексту пошуку, який не бере участі у реляційних операціях і не порушує принципу атомарності в розумінні класичної реляційної моделі.

**Друга нормальна форма (2НФ).** Відношення знаходиться у другій нормальній формі, якщо воно знаходиться у 1НФ і всі його неключові атрибути функціонально повно залежать від первинного ключа, тобто не залежать лише від частини складеного ключа. У розробленій базі даних кожне відношення має простий первинний ключ типу INT (car\_id, client\_id, employee\_id, request\_id, order\_id, tracking\_id), що автоматично гарантує виконання умови 2НФ. Усі

неключові атрибути кожної таблиці залежать виключно від первинного ключа цієї таблиці.

**Третя нормальна форма (3НФ).** Відношення знаходиться у третій нормальній формі, якщо воно знаходиться у 2НФ і жоден неключовий атрибут не залежить транзитивно від первинного ключа через інший неключовий атрибут. Усі таблиці розробленої бази даних задовольняють цю вимогу. Зокрема, у таблиці Orders інформація про клієнта не дублюється повторно (ім'я, телефон, email клієнта зберігаються лише у таблиці Clients і доступні через зовнішній ключ `client_id`), що повністю усуває транзитивні залежності. Аналогічно, інформація про автомобіль (марка, модель, ціна тощо) зберігається лише у таблиці Cars і не дублюється у заявках та замовленнях.

Окрім дотримання нормальних форм, у схемі бази даних свідомо передбачено два поля, які могли б розглядатися як надмірні: `client_name` і `client_phone` у таблиці Requests. Ці поля дублюють інформацію, яку можна отримати через зв'язок з таблицею Clients. Однак це рішення є усвідомленою денормалізацією, що дозволяє приймати заявки від незареєстрованих відвідувачів сайту (з ручним введенням контактних даних у форму) та зберігати ці заявки навіть у випадку, якщо клієнт згодом видалить свій обліковий запис. Таким чином, заявки залишаються інформативними навіть без активного зв'язку з таблицею Clients.

Внаслідок проведеної нормалізації структура бази даних є оптимальною, забезпечує цілісність даних, мінімізує ризик аномалій оновлення та значно полегшує супровід інформаційної системи. Кожна сутність предметної області представлена окремою таблицею з чітко визначеним переліком атрибутів, а інформаційні взаємозв'язки реалізовані через зовнішні ключі.

### **2.3. Визначення типів даних**

Правильний вибір типу даних для кожного атрибута бази даних безпосередньо впливає на цілісність інформації, ефективність використання дискового простору, швидкість виконання SQL-запитів та коректність обробки

значень на рівні застосунку. У мові SQL (зокрема, в діалекті MySQL) передбачено три основні групи типів даних: текстові, числові та такі, що пов'язані з часовими позначками. Для забезпечення обґрунтованого вибору розглянемо кожен з цих груп.

Таблиця 2.1

### Текстові типи даних

| Назва      | Опис                                                                                                    |
|------------|---------------------------------------------------------------------------------------------------------|
| CHAR(n)    | Рядок фіксованої довжини n символів. Невикористані позиції доповнюються пробілами.                      |
| VARCHAR(n) | Рядок змінної довжини до n символів. Використовує лише стільки місця, скільки потрібно.                 |
| TEXT       | Рядок з максимальною довжиною 65 535 символів. Призначений для зберігання великих текстових фрагментів. |
| ENUM       | Перерахування. Дозволяє вибрати одне значення зі списку, визначеного при створенні таблиці.             |
| JSON       | Спеціалізований тип для зберігання структурованих даних у форматі JSON з підтримкою індексів і функцій. |

У базі даних автосалону текстові типи представлені переважно у вигляді VARCHAR і ENUM. Поля VARCHAR використовуються для зберігання імен (first\_name, last\_name з обмеженням 50 символів), назв марок і моделей автомобілів (make, model), VIN-кодів (фіксована довжина 17 символів), країн походження, номерів телефонів, електронних адрес (email VARCHAR(100)) та хешів паролів (password VARCHAR(255), що достатньо для будь-якого алгоритму хешування). Тип ENUM застосовано для атрибутів, які приймають значення зі строго обмеженого переліку: тип палива, тип трансмісії, тип приводу, статус автомобіля, статус замовлення, тип клієнта, посада працівника, статус заявки, статус оплати. Такий підхід забезпечує контроль допустимих значень безпосередньо на рівні бази даних і виключає введення некоректних даних. Поле search\_context у таблиці Requests має тип JSON, що дозволяє гнучко зберігати різноманітні параметри пошуку без необхідності модифікувати схему бази даних при появі нових критеріїв.

**Числові типи даних**

| Назва        | Опис                                                                                             |
|--------------|--------------------------------------------------------------------------------------------------|
| INT          | Цілі числа в діапазоні від -2 147 483 648 до 2 147 483 647. Займає 4 байти.                      |
| TINYINT      | Малий цілий тип з діапазоном від -128 до 127. Часто використовується як логічний тип.            |
| BIGINT       | Великі цілі числа в діапазоні до $2^{63}$ . Займає 8 байтів.                                     |
| DECIMAL(M,D) | Числа з фіксованою точністю та масштабом. M - загальна кількість цифр, D - кількість після коми. |
| FLOAT        | Числа з плаваючою комою одинарної точності (4 байти). Може мати похибки округлення.              |
| DOUBLE       | Числа з плаваючою комою подвійної точності (8 байтів). Підвищена точність.                       |

Числові типи у базі даних автосалону представлені, насамперед, типом INT, який застосовується для всіх первинних і зовнішніх ключів (car\_id, client\_id, employee\_id, request\_id, order\_id, tracking\_id), а також для атрибута пробігу автомобіля (odometer). Для зберігання грошових сум використано тип DECIMAL з належно підбраною точністю: purchase\_price і selling\_price оголошено як DECIMAL(10,2), що дозволяє зберігати суми до 99 999 999.99, а total\_amount у таблиці Orders - як DECIMAL(15,2) для можливості обліку особливо дорогих замовлень. Тип DECIMAL обрано замість FLOAT/DOUBLE спеціально для роботи з фінансовими сумами, оскільки він гарантує точне зберігання значень без похибок округлення, що є критично важливим у бухгалтерських розрахунках. Об'єм двигуна оголошено як DECIMAL(3,1) (наприклад, 2.0, 3.5, 6.2 літра).

## Типи даних для дати та часу

| Назва     | Опис                                                            |
|-----------|-----------------------------------------------------------------|
| DATE      | Зберігає дату у форматі РРРР-ММ-ДД.                             |
| TIME      | Зберігає час у форматі ГГ:ХХ:СС.                                |
| DATETIME  | Зберігає дату й час у форматі РРРР-ММ-ДД ГГ:ХХ:СС.              |
| TIMESTAMP | Часова мітка. Підтримує автоматичне оновлення при зміні запису. |
| YEAR      | Зберігає рік у форматі РРРР (4 цифри).                          |

Типи даних, пов'язані з датами, у базі даних автосалону використовуються таким чином: тип YEAR - для року випуску автомобіля (manufacture\_year); тип DATE - для дати укладення договору (contract\_date), очікуваної дати прибуття (estimated\_arrival) та дати прийому на роботу працівника (hire\_date); тип DATETIME - для дати подання заявки (request\_date) і дати події логістики (status\_date), оскільки в цих випадках важливим є не лише календарний день, але й точний час; тип TIMESTAMP - для службових часових міток created\_at у таблицях Cars і Clients з автоматичною підстановкою поточного часу при створенні запису. [3, 11]

## 2.4. Обмеження цілісності даних

Обмеження цілісності даних - це механізми, які гарантують відповідність даних, що зберігаються у базі, певним заздалегідь визначеним правилам та логічним умовам предметної області. Завдяки обмеженням цілісності уникнути ситуацій, коли у базі з'являються некоректні, суперечливі або «осиротілі» записи, що порушують цілісність інформаційної системи. У розробленій базі даних автосалону застосовано декілька типів обмежень цілісності. [11]

**Первинні ключі (Primary Keys).** Первинний ключ забезпечує унікальну ідентифікацію кожного запису в таблиці. У всіх шести таблицях бази даних автосалону первинні ключі реалізовано як цілочислові поля з атрибутом AUTO\_INCREMENT, що гарантує автоматичну генерацію унікальних значень при додаванні нових записів. Зокрема, у таблиці Cars первинним ключем є car\_id;

у таблиці Clients - client\_id; у таблиці Employees - employee\_id; у таблиці Requests - request\_id; у таблиці Orders - order\_id; у таблиці Delivery\_Tracking - tracking\_id.

**Зовнішні ключі (Foreign Keys).** Зовнішні ключі реалізують міжатабличні зв'язки та гарантують посилальну цілісність -неможливість існування записів, що посилаються на неіснуючі записи в інших таблицях. У розробленій базі даних таблиця Requests має зовнішні ключі: client\_id (посилається на Clients), car\_id (на Cars) та employee\_id (на Employees). Таблиця Orders має чотири зовнішні ключі: client\_id, employee\_id, car\_id, request\_id. Таблиця Delivery\_Tracking пов'язана з Orders через order\_id. Завдяки цим обмеженням неможливо створити, наприклад, замовлення для неіснуючого клієнта або заявку на автомобіль, якого немає в каталозі.

**Унікальні ключі (Unique Constraints).** Обмеження UNIQUE гарантує, що значення певного атрибута не повторюється в межах таблиці. У базі даних автосалону таке обмеження застосовано до полів email у таблицях Clients та Employees (оскільки електронна адреса слугує логіном при авторизації), а також до поля vin\_code у таблиці Cars (оскільки VIN-код є унікальним ідентифікатором кожного транспортного засобу у світовій практиці).

**Обмеження на значення атрибутів (CHECK / ENUM).** Контроль допустимих значень здійснюється переважно за допомогою типу ENUM, який обмежує можливі значення атрибута заздалегідь визначеним переліком. Так, поле fuel\_type може приймати лише значення «бензин», «дизель», «газ», «гібрид» або «електро»; поле transmission - «автомат» або «механіка»; поле car\_status - «заброньовано», «на аукціоні», «в дорозі», «в ремонті», «в наявності» або «продано». Аналогічні обмеження накладено на поля status у таблиці Requests, order\_status і payment\_status у таблиці Orders, position у таблиці Employees, client\_type у таблиці Clients.

**Обов'язковість значень (NOT NULL).** Ключові поля, які повинні бути обов'язково заповнені при створенні запису (ім'я, прізвище, email, телефон, марка автомобіля, VIN-код, дати тощо), оголошено з обмеженням NOT NULL.



Це гарантує, що в базі не з'являться записи з порожніми обов'язковими полями, що могло б призвести до некоректної роботи інформаційної системи.

**Автозбільшення (AUTO\_INCREMENT).** Атрибут AUTO\_INCREMENT застосовано до всіх первинних ключів. При додаванні нового запису СКБД автоматично присвоює полю значення, на одиницю більше за максимальне існуюче. Це повністю усуває ризик колізії ключів при паралельному додаванні записів і спрощує роботу з базою даних. [2]

Сукупність застосованих обмежень цілісності забезпечує високий рівень надійності бази даних автосалону, гарантує логічну узгодженість інформації та запобігає більшості типових помилок, які можуть виникнути при ручному введенні даних або під час паралельної роботи декількох користувачів системи.

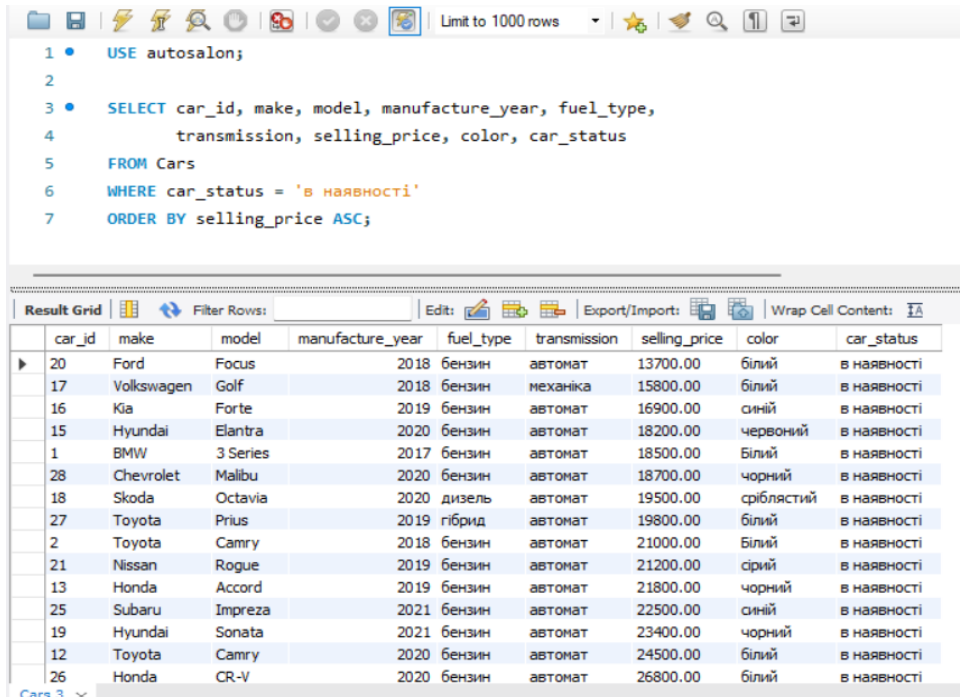
## **2.5. Реалізація SQL-скрипту**

SQL-скрипт - це послідовність SQL-команд, що виконуються одна за одною для створення структури бази даних, наповнення її даними, виконання запитів на вибірку та модифікацію інформації. Для розробки бази даних автосалону використано середовище MySQL Workbench, у якому було спроектовано EER-діаграму, згенеровано SQL-скрипт створення таблиць (CREATE TABLE) та виконано тестове наповнення бази даних.

У процесі розробки інформаційної системи були реалізовані численні SQL-запити, що відповідають основним функціональним вимогам системи. Розглянемо найважливіші з них.

**Перегляд каталогу автомобілів у наявності.** Цей запит лежить в основі публічної сторінки каталогу та формує перелік автомобілів, доступних для негайного продажу клієнтам. Він повертає основні характеристики кожного авто - марку, модель, рік випуску, тип палива, тип трансмісії, колір та ціну - які виводяться у вигляді карток на сторінці каталогу. Фільтрація за статусом 'в наявності' гарантує, що клієнти бачать виключно ті автомобілі, які реально можна купити, тоді як авто зі статусами «в дорозі», «на аукціоні», «заброньовано» чи «продано» залишаються прихованими. Сортування за ціною

за зростанням дозволяє клієнту швидко зорієнтуватися у ціновому діапазоні пропозиції (рис. 2.2).



The screenshot shows a database query interface. At the top, there is a toolbar with icons for file operations, search, and execution. Below the toolbar, a SQL query is entered in a text area:

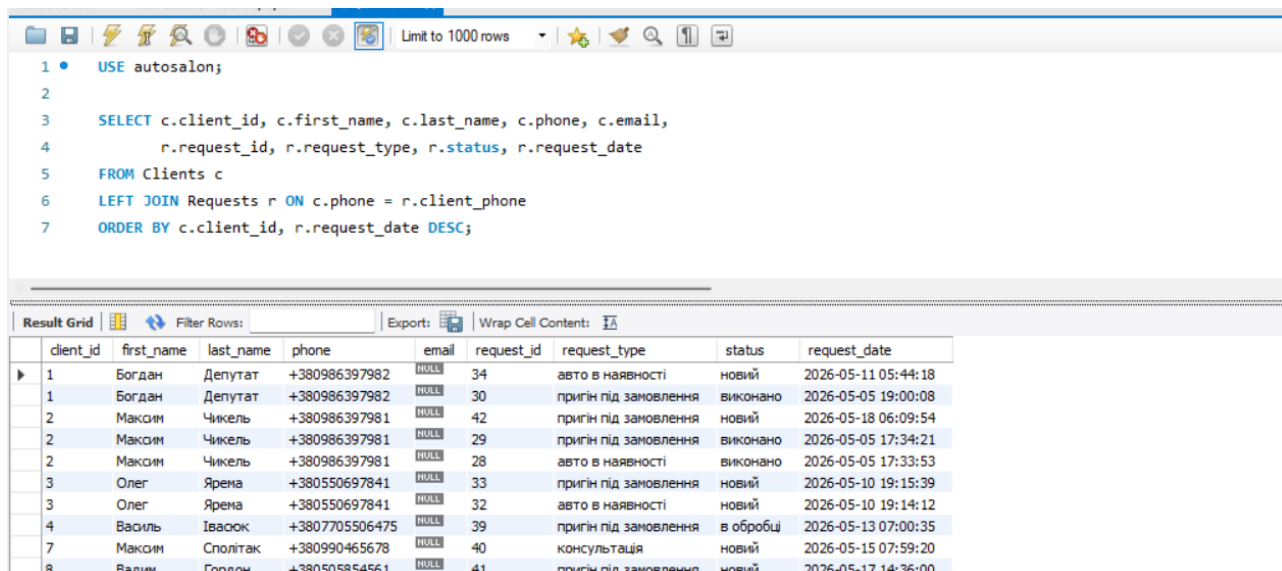
```
1 • USE autosalon;  
2  
3 • SELECT car_id, make, model, manufacture_year, fuel_type,  
4         transmission, selling_price, color, car_status  
5 FROM Cars  
6 WHERE car_status = 'в наявності'  
7 ORDER BY selling_price ASC;
```

Below the query, there is a "Result Grid" section. It contains a table with 10 columns: car\_id, make, model, manufacture\_year, fuel\_type, transmission, selling\_price, color, and car\_status. The table displays 16 rows of car data, sorted by selling price in ascending order. The status for all cars is "в наявності".

| car_id | make       | model    | manufacture_year | fuel_type | transmission | selling_price | color      | car_status  |
|--------|------------|----------|------------------|-----------|--------------|---------------|------------|-------------|
| 20     | Ford       | Focus    | 2018             | бензин    | автомат      | 13700.00      | білий      | в наявності |
| 17     | Volkswagen | Golf     | 2018             | бензин    | механіка     | 15800.00      | білий      | в наявності |
| 16     | Kia        | Forte    | 2019             | бензин    | автомат      | 16900.00      | синій      | в наявності |
| 15     | Hyundai    | Elantra  | 2020             | бензин    | автомат      | 18200.00      | червоний   | в наявності |
| 1      | BMW        | 3 Series | 2017             | бензин    | автомат      | 18500.00      | білий      | в наявності |
| 28     | Chevrolet  | Malibu   | 2020             | бензин    | автомат      | 18700.00      | чорний     | в наявності |
| 18     | Skoda      | Octavia  | 2020             | дизель    | автомат      | 19500.00      | сріблястий | в наявності |
| 27     | Toyota     | Prius    | 2019             | гібрид    | автомат      | 19800.00      | білий      | в наявності |
| 2      | Toyota     | Camry    | 2018             | бензин    | автомат      | 21000.00      | білий      | в наявності |
| 21     | Nissan     | Rogue    | 2019             | бензин    | автомат      | 21200.00      | срібний    | в наявності |
| 13     | Honda      | Accord   | 2019             | бензин    | автомат      | 21800.00      | чорний     | в наявності |
| 25     | Subaru     | Impreza  | 2021             | бензин    | автомат      | 22500.00      | синій      | в наявності |
| 19     | Hyundai    | Sonata   | 2021             | бензин    | автомат      | 23400.00      | чорний     | в наявності |
| 12     | Toyota     | Camry    | 2020             | бензин    | автомат      | 24500.00      | білий      | в наявності |
| 26     | Honda      | CR-V     | 2020             | бензин    | автомат      | 26800.00      | білий      | в наявності |

**Рис. 2.2. Перегляд каталогу автомобілів у наявності**

Виведення повної інформації про клієнтів та їхні заявки. Запит дозволяє отримати дані про зареєстрованих клієнтів автосалону разом з історією поданих ними заявок. Він використовується менеджерами в адміністративній панелі для роботи з клієнтською базою. Особливістю реалізації є те, що заявки шукаються не за прямим зовнішнім ключем `client_id`, а за полем `client_phone`. Це усвідомлене проектне рішення, прийняте на етапі моделювання бази даних: воно дозволяє пов'язувати з клієнтом навіть ті заявки, які були подані до його реєстрації у системі. Адже клієнт, що залишив заявку як незареєстрований відвідувач, після створення облікового запису повинен побачити свою стару заявку в особистому кабінеті - за умови, що номер телефону залишився тим самим. Таким чином, телефон виступає природним «зв'язком» між анонімними заявками і обліковими записами клієнтів (рис. 2.3).



Limit to 1000 rows

```

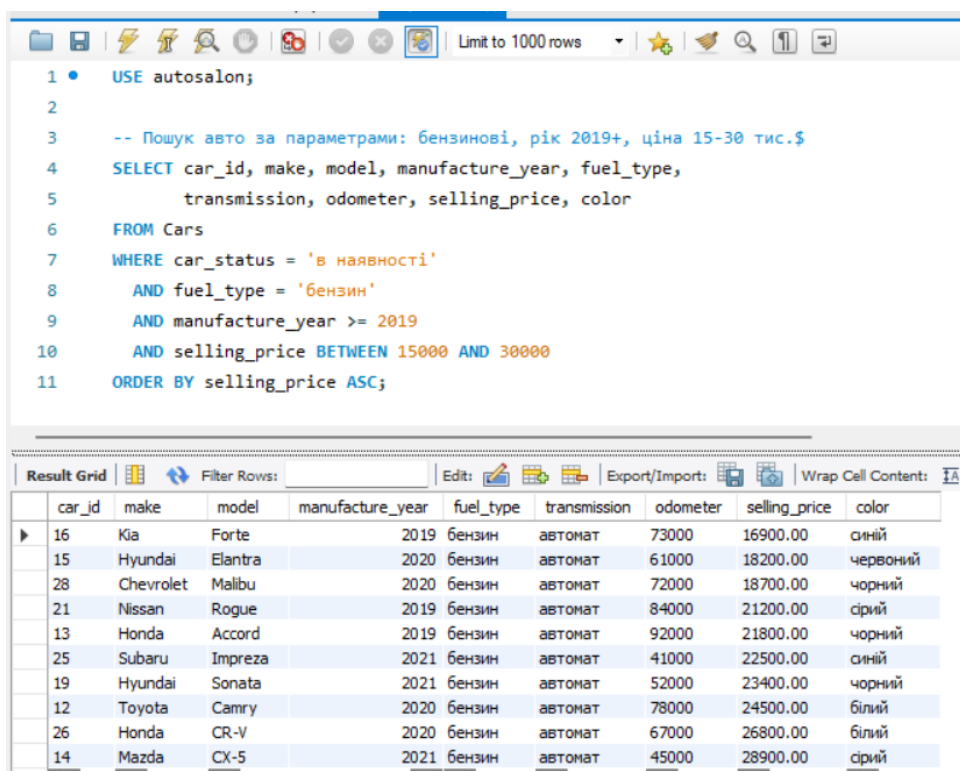
1 • USE autosalon;
2
3 SELECT c.client_id, c.first_name, c.last_name, c.phone, c.email,
4        r.request_id, r.request_type, r.status, r.request_date
5 FROM Clients c
6 LEFT JOIN Requests r ON c.phone = r.client_phone
7 ORDER BY c.client_id, r.request_date DESC;

```

|   | client_id | first_name | last_name | phone          | email | request_id | request_type          | status    | request_date        |
|---|-----------|------------|-----------|----------------|-------|------------|-----------------------|-----------|---------------------|
| ▶ | 1         | Богдан     | Депутат   | +380986397982  | NULL  | 34         | авто в наявності      | новий     | 2026-05-11 05:44:18 |
|   | 1         | Богдан     | Депутат   | +380986397982  | NULL  | 30         | пригін під замовлення | виконано  | 2026-05-05 19:00:08 |
|   | 2         | Максим     | Чикель    | +380986397981  | NULL  | 42         | пригін під замовлення | новий     | 2026-05-18 06:09:54 |
|   | 2         | Максим     | Чикель    | +380986397981  | NULL  | 29         | пригін під замовлення | виконано  | 2026-05-05 17:34:21 |
|   | 2         | Максим     | Чикель    | +380986397981  | NULL  | 28         | авто в наявності      | виконано  | 2026-05-05 17:33:53 |
|   | 3         | Олег       | Ярена     | +380550697841  | NULL  | 33         | пригін під замовлення | новий     | 2026-05-10 19:15:39 |
|   | 3         | Олег       | Ярена     | +380550697841  | NULL  | 32         | авто в наявності      | новий     | 2026-05-10 19:14:12 |
|   | 4         | Василь     | Івасюк    | +3807705506475 | NULL  | 39         | пригін під замовлення | в обробці | 2026-05-13 07:00:35 |
|   | 7         | Максим     | Сполітак  | +380990465678  | NULL  | 40         | консультація          | новий     | 2026-05-15 07:59:20 |
|   | 8         | Вадим      | Гордон    | +380505854561  | NULL  | 41         | пригін під замовлення | новий     | 2026-05-17 14:36:00 |

**Рис. 2.3. Клієнти автосалону та їхні заявки**

**Пошук автомобілів за параметрами фільтрації.** У наведеному прикладі клієнт шукає бензиновий автомобіль 2019 року випуску або новіший у ціновому діапазоні від 15 000 до 30 000 доларів США. Сортування за ціною за зростанням є стандартним для каталогів електронної комерції - клієнт спочатку бачить найдешевші варіанти, що відповідають його критеріям (рис. 2.4).



Limit to 1000 rows

```

1 • USE autosalon;
2
3 -- Пошук авто за параметрами: бензинові, рік 2019+, ціна 15-30 тис.$
4 SELECT car_id, make, model, manufacture_year, fuel_type,
5        transmission, odometer, selling_price, color
6 FROM Cars
7 WHERE car_status = 'в наявності'
8       AND fuel_type = 'бензин'
9       AND manufacture_year >= 2019
10      AND selling_price BETWEEN 15000 AND 30000
11 ORDER BY selling_price ASC;

```

|   | car_id | make      | model   | manufacture_year | fuel_type | transmission | odometer | selling_price | color    |
|---|--------|-----------|---------|------------------|-----------|--------------|----------|---------------|----------|
| ▶ | 16     | Kia       | Forte   | 2019             | бензин    | автомат      | 73000    | 16900.00      | синій    |
|   | 15     | Hyundai   | Elantra | 2020             | бензин    | автомат      | 61000    | 18200.00      | червоний |
|   | 28     | Chevrolet | Malibu  | 2020             | бензин    | автомат      | 72000    | 18700.00      | чорний   |
|   | 21     | Nissan    | Rogue   | 2019             | бензин    | автомат      | 84000    | 21200.00      | сірий    |
|   | 13     | Honda     | Accord  | 2019             | бензин    | автомат      | 92000    | 21800.00      | чорний   |
|   | 25     | Subaru    | Impreza | 2021             | бензин    | автомат      | 41000    | 22500.00      | синій    |
|   | 19     | Hyundai   | Sonata  | 2021             | бензин    | автомат      | 52000    | 23400.00      | чорний   |
|   | 12     | Toyota    | Camry   | 2020             | бензин    | автомат      | 78000    | 24500.00      | білий    |
|   | 26     | Honda     | CR-V    | 2020             | бензин    | автомат      | 67000    | 26800.00      | білий    |
|   | 14     | Mazda     | CX-5    | 2021             | бензин    | автомат      | 45000    | 28900.00      | сірий    |

**Рис. 2.4. Фільтрація авто за параметрами**

Список менеджерів автосалону з кількістю оброблених заявок. (рис. 2.5).

```

2
3  -- Список менеджерів автосалону з кількістю оброблених заявок
4  SELECT e.employee_id,
5         CONCAT(e.first_name, ' ', e.last_name) AS manager,
6         e.position,
7         COUNT(r.request_id) AS processed_requests
8  FROM Employees e
9  LEFT JOIN Requests r ON e.employee_id = r.employee_id
10 WHERE e.position LIKE '%Менеджер%'
11 GROUP BY e.employee_id, e.first_name, e.last_name, e.position
12 ORDER BY processed_requests DESC;

```

| employee_id | manager          | position                  | processed_requests |
|-------------|------------------|---------------------------|--------------------|
| 4           | Олег Винник      | Менеджер з пригону        | 8                  |
| 2           | Андрій Шевченко  | Менеджер авто в наявності | 5                  |
| 3           | Марина Коваленко | Менеджер авто в наявності | 0                  |
| 6           | Назар Хвесик     | Менеджер з пригону        | 0                  |

Рис. 2.5. Менеджери та їхні заявки

Витягнення повної інформації про замовлення з усіма пов'язаними сутностями. Цей запит реалізує одну з ключових функцій адміністративної панелі - відображення таблиці замовлень з повним контекстом для кожного запису. (рис. 2.6).

```

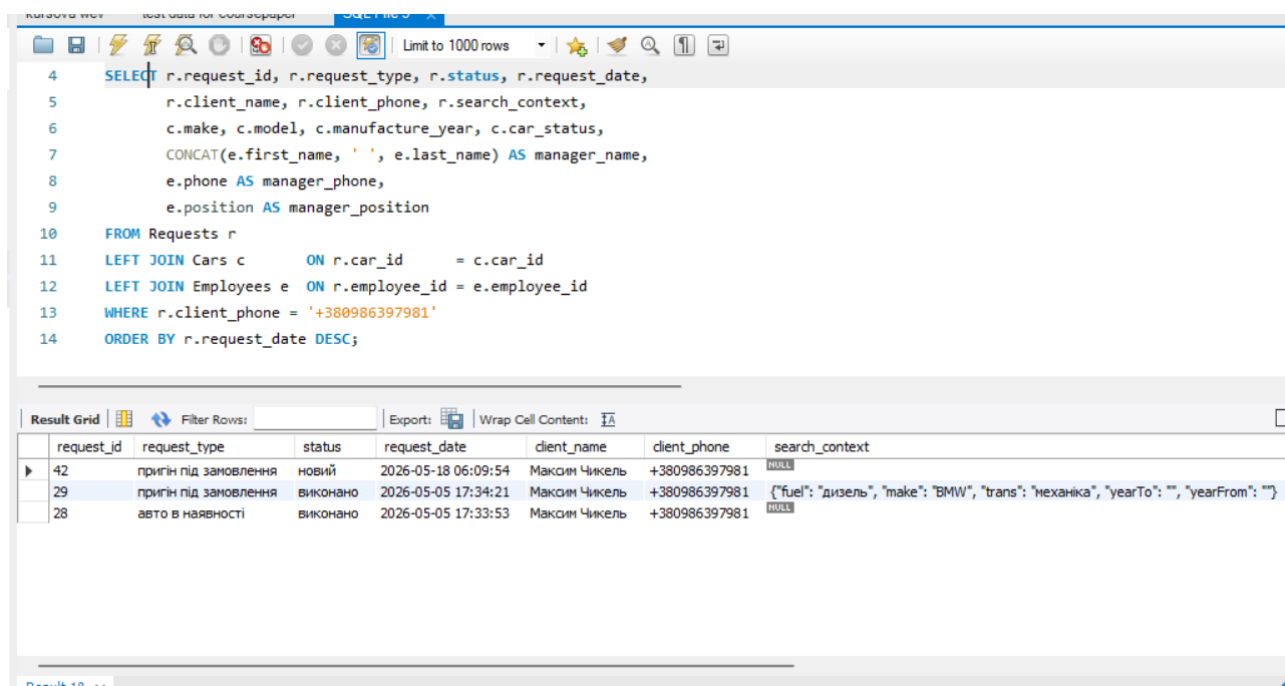
4 • SELECT o.order_id, o.contract_date, o.total_amount,
5         o.payment_status, o.order_status,
6         c.first_name AS client_first, c.last_name AS client_last,
7         c.phone AS client_phone,
8         car.make, car.model, car.vin_code, car.car_status,
9         emp.first_name AS manager_first, emp.last_name AS manager_last
10 FROM Orders o
11 LEFT JOIN Clients c ON o.client_id = c.client_id
12 LEFT JOIN Cars car ON o.car_id = car.car_id
13 LEFT JOIN Employees emp ON o.employee_id = emp.employee_id
14 ORDER BY o.contract_date DESC;

```

| order_id | contract_date | total_amount | payment_status    | order_status | client_first | client_last | client_phone   | make   | model          | vin_code          | car_sta |
|----------|---------------|--------------|-------------------|--------------|--------------|-------------|----------------|--------|----------------|-------------------|---------|
| 4        | 2026-05-17    | 1000.00      | частково сплачено | новий        | Вадим        | Гордон      | +380505854561  | BMW    | 3 series       | 2FMPK4J80JBB00002 | забронь |
| 3        | 2026-05-13    | 3000.00      | частково сплачено | в процесі    | Василь       | Івасюк      | +3807705506475 | Ford   | Edge           | 2FMPK4J80JBB00001 | на аукц |
| 10       | 2026-05-10    | 33800.00     | частково сплачено | в процесі    | Юлія         | Бондаренко  | +380501234506  | Ford   | Mustang        | 1FA6P8TH1J5492501 | забронь |
| 1        | 2026-05-05    | 3000.00      | частково сплачено | в процесі    | Макоїн       | Чикель      | +380986397981  | Toyota | Camry          | 2T1BURHE0JC043821 | в наявн |
| 2        | 2026-05-05    | 2000.00      | частково сплачено | новий        | Богдан       | Депутат     | +380986397982  | Jeep   | Grand Cherokee | 1G1ZT53806F109719 | в дороз |
| 9        | 2026-04-28    | 35800.00     | оплачено          | завершено    | Андрій       | Мельник     | +380501234505  | Mazda  | CX-9           | JM3TCBDY9K0492305 | продан  |
| 8        | 2026-04-10    | 33500.00     | оплачено          | завершено    | Наталія      | Шевченко    | +380501234504  | Honda  | Pilot          | 5J6RW2H58JL492304 | продан  |
| 7        | 2026-03-22    | 31200.00     | оплачено          | завершено    | Тарас        | Коваленко   | +380501234503  | BMW    | 3 Series       | WBAJA5C5XJ7492303 | продан  |
| 6        | 2026-03-08    | 18900.00     | оплачено          | завершено    | Марія        | Іваненко    | +380501234502  | Honda  | Civic          | 1HGCY1F30JA492302 | продан  |

Рис. 2.6. Замовлення з повними даними

**Отримання заявок конкретного клієнта для відображення в особистому кабінеті.** На вкладці «Мої заявки» клієнт бачить повну історію своїх звернень до автосалону, з поточним статусом кожної заявки, інформацією про автомобіль (якщо заявка стосувалася конкретного авто з каталогу) та контактами менеджера, який її опрацьовує. (рис.2.7).



The screenshot shows a database query editor with a SQL query and its results in a grid. The query is as follows:

```

4  SELECT r.request_id, r.request_type, r.status, r.request_date,
5         r.client_name, r.client_phone, r.search_context,
6         c.make, c.model, c.manufacture_year, c.car_status,
7         CONCAT(e.first_name, ' ', e.last_name) AS manager_name,
8         e.phone AS manager_phone,
9         e.position AS manager_position
10 FROM Requests r
11 LEFT JOIN Cars c ON r.car_id = c.car_id
12 LEFT JOIN Employees e ON r.employee_id = e.employee_id
13 WHERE r.client_phone = '+380986397981'
14 ORDER BY r.request_date DESC;

```

The results are displayed in a grid with the following columns: request\_id, request\_type, status, request\_date, client\_name, client\_phone, and search\_context. The data is as follows:

| request_id | request_type          | status   | request_date        | client_name   | client_phone  | search_context                                                                       |
|------------|-----------------------|----------|---------------------|---------------|---------------|--------------------------------------------------------------------------------------|
| 42         | пригін під замовлення | новий    | 2026-05-18 06:09:54 | Максим Чикель | +380986397981 | NULL                                                                                 |
| 29         | пригін під замовлення | виконано | 2026-05-05 17:34:21 | Максим Чикель | +380986397981 | {"fuel": "дизель", "make": "BMW", "trans": "механіка", "yearTo": "", "yearFrom": ""} |
| 28         | авто в наявності      | виконано | 2026-05-05 17:33:53 | Максим Чикель | +380986397981 | NULL                                                                                 |

**Рис. 2.7. Заявки клієнта в особистому кабінеті**

**Відстеження хронології доставки автомобілів.** Цей запит реалізує одну з найбільш цінних для клієнтів функцій інформаційної системи - прозоре відстеження логістики доставки замовленого автомобіля. У результатуючій вибірці на одному рядку відображається кожна подія переміщення авто - від моменту купівлі на аукціоні до прибуття на склад автосалону. Кожний запис містить поточне розташування, текстовий опис статусу, точну дату події та орієнтовну дату прибуття. (рис. 2.8).

The screenshot shows a SQL IDE window with a query and its results. The query joins Delivery\_Tracking, Orders, and Clients tables. The result grid displays columns: tracking\_id, current\_location, status\_description, status\_date, estimated\_arrival, client, client\_phone, and car.

| tracking_id | current_location                | status_description                            | status_date         | estimated_arrival | client          | client_phone   | car          |
|-------------|---------------------------------|-----------------------------------------------|---------------------|-------------------|-----------------|----------------|--------------|
| 3           | Аукціон копарт                  | Виграли лот                                   | 2026-05-17 00:00:00 | 2026-09-24        | Вадим Гордон    | +380505854561  | BMW 3 series |
| 6           | Атлантичний океан               | Судно прямує до порту Гданськ                 | 2026-05-16 08:00:00 | 2026-06-20        | Юлія Бондаренко | +380501234506  | Ford Mustan  |
| 2           | Порт Одеса                      | Завантажено на автовоз                        | 2026-05-13 00:00:00 | 2026-05-16        | Василь Івасок   | +3807705506475 | Ford Edge (  |
| 5           | Порт Балтимор, США              | Автомобіль завантажено на судно               | 2026-05-12 14:30:00 | 2026-06-25        | Юлія Бондаренко | +380501234506  | Ford Mustan  |
| 4           | Аукціон Copart, Нью-Джерсі, США | Авто виграє на аукціоні, готується до відп... | 2026-05-10 10:00:00 | 2026-06-25        | Юлія Бондаренко | +380501234506  | Ford Mustan  |
| 1           | Порт Балтимор                   | Завантажено на корабель                       | 2026-05-06 00:00:00 | 2026-06-17        | Богдан Депутат  | +380986397982  | Jeep Grand   |

**Рис. 2.8. Хронологія доставки авто**

Зведена статистика для дашборду адміністратора. Цей запит формує всі ключові показники діяльності автосалону в одному запиті - кількість авто за статусами, загальну кількість клієнтів, кількість заявок у різних станах обробки, кількість активних замовлень та сумарний дохід від виконаних угод. (рис. 2.9).

The screenshot shows a SQL IDE window with a summary statistics query. The query uses COUNT and SUM functions to aggregate data from Cars, Clients, Requests, and Orders tables. The result grid displays columns: cars\_available, cars\_in\_transit, cars\_sold, total\_clients, new\_requests, in\_progress\_requests, active\_orders, and total\_revenue.

| cars_available | cars_in_transit | cars_sold | total_clients | new_requests | in_progress_requests | active_orders | total_revenue |
|----------------|-----------------|-----------|---------------|--------------|----------------------|---------------|---------------|
| 20             | 15              | 6         | 16            | 11           | 2                    | 3             | 134800.00     |

**Рис. 2.9. Зведена статистика автосалону**

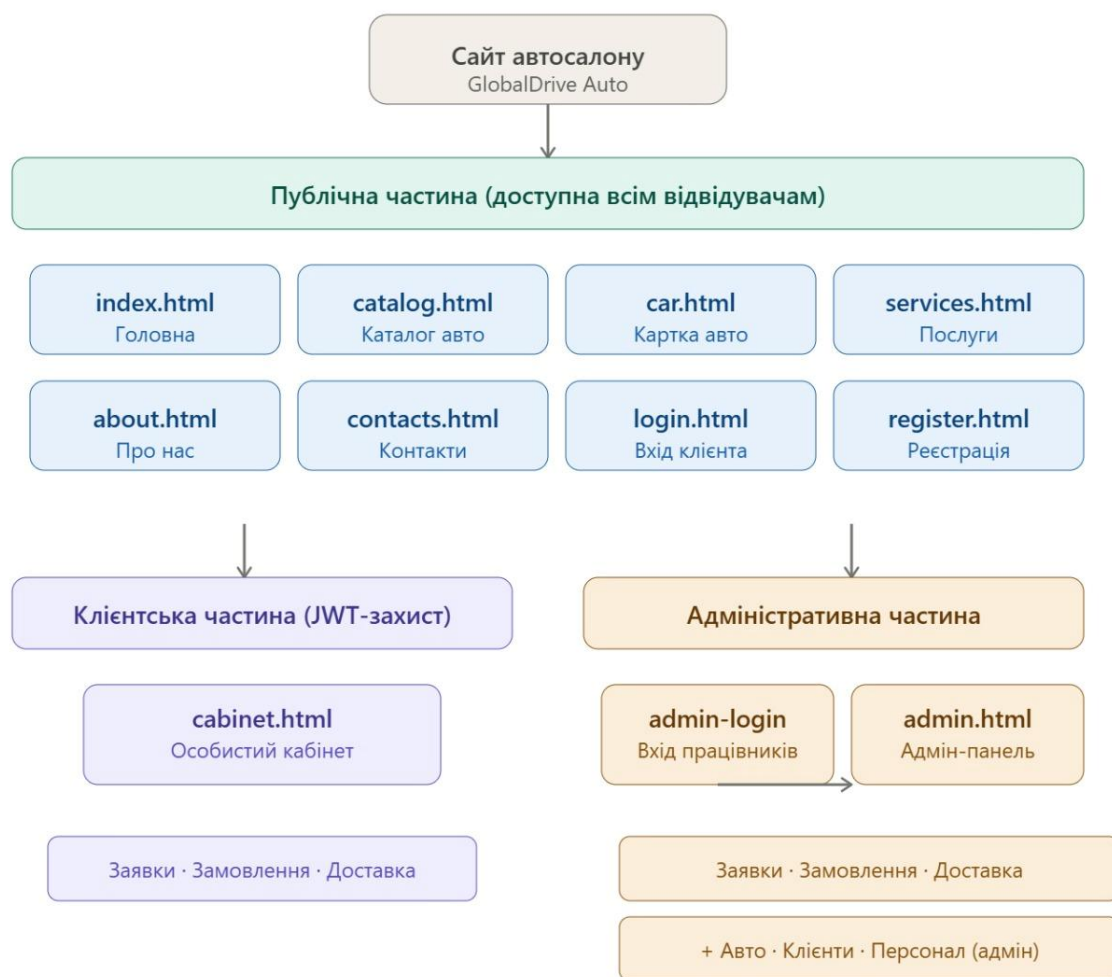
Усі наведені вище SQL-запити використовуються у складі серверної частини інформаційної системи -кожен з них виконується через бібліотеку mysql2 у відповідь на запити клієнтської частини додатку. Повний лістинг SQL-запитів, що реалізують основні функціональні можливості системи, наведено у Додатку Б.



## РОЗДІЛ 3. РОЗРОБКА ВЕБ-ДОДАТКУ ДЛЯ АВТОСАЛОНУ

### 3.1. Структура веб-сайту

Структура веб-додатку інформаційної системи автосалону «GlobalDrive Auto» спроектована з урахуванням потреб трьох категорій користувачів: відвідувачів сайту (потенційних клієнтів), зареєстрованих клієнтів та працівників автосалону (менеджерів і адміністраторів). Кожна категорія користувачів має доступ до певного набору сторінок, що відповідає її функціональним повноваженням. Загальна карта сайту представлена на рис. 3.1.



**Рис. 3.1.** Карта сайту автосалону

*Джерело: Розроблено автором*

Публічна частина сайту, доступна всім без винятку відвідувачам, складається з таких сторінок:

- index.html -головна сторінка сайту з презентаційним блоком, описом переваг автосалону та закликом до дії;
- catalog.html -каталог автомобілів з можливостями пошуку та фільтрації за маркою, моделлю, роком випуску, типом палива, коробкою передач, ціновим діапазоном;
- car.html -детальна сторінка конкретного автомобіля з повним описом характеристик, фотографією та формою подання заявки на пригін чи консультацію;
- services.html -опис послуг автосалону, які включають підбір авто на закордонних аукціонах, перевірку технічного стану, доставку, митне оформлення та реєстрацію;
- about.html -інформація про автосалон, його місію, переваги співпраці та команду;
- contacts.html -контактна інформація: адреса, телефони, графік роботи та форма зворотного зв'язку;
- login.html -сторінка авторизації для клієнтів та працівників;
- register.html -сторінка реєстрації нового клієнта.

Для зареєстрованих клієнтів додатково доступна сторінка cabinet.html - особистий кабінет, де клієнт може переглянути власні заявки з їхніми поточними статусами, побачити інформацію про укладені договори, відстежити доставку замовленого автомобіля у режимі реального часу та залишити відгук про роботу автосалону. Доступ до особистого кабінету захищено за допомогою JWT-токена авторизації; неавторизовані користувачі автоматично перенаправляються на сторінку входу.

Для входу працівників автосалону передбачено окрему сторінку admin-login.html, яка візуально та функціонально відокремлена від клієнтської форми авторизації. Розділення сторінок входу для клієнтів і працівників є усвідомленим архітектурним рішенням: воно дозволяє приховати службовий вхід від звичайних відвідувачів сайту та використовувати різні форми, повідомлення про помилки та правила валідації для різних категорій користувачів.



Адміністративна частина системи реалізована у вигляді єдиної сторінки `admin.html`, до якої мають доступ виключно працівники автосалону. На основі ролі (адміністратор або менеджер), визначеної у JWT-токені, на стороні клієнта виконується відповідне налаштування інтерфейсу: менеджеру відображаються розділи роботи з заявками, замовленнями та доставкою, тоді як адміністратору додатково доступні розділи керування каталогом автомобілів, персоналом та клієнтською базою. Така архітектура дозволяє при додаванні нових ролей у майбутньому не створювати окремі сторінки, а лише розширювати функціональні можливості єдиного адміністративного інтерфейсу.

Окрім перерахованих сторінок, у структурі сайту присутні допоміжні файли: каталоги `css/` зі стилями оформлення, `js/` зі скриптами клієнтської частини та `images/` зі статичними зображеннями (логотип, фотографії автомобілів, ілюстративні зображення для головної сторінки).

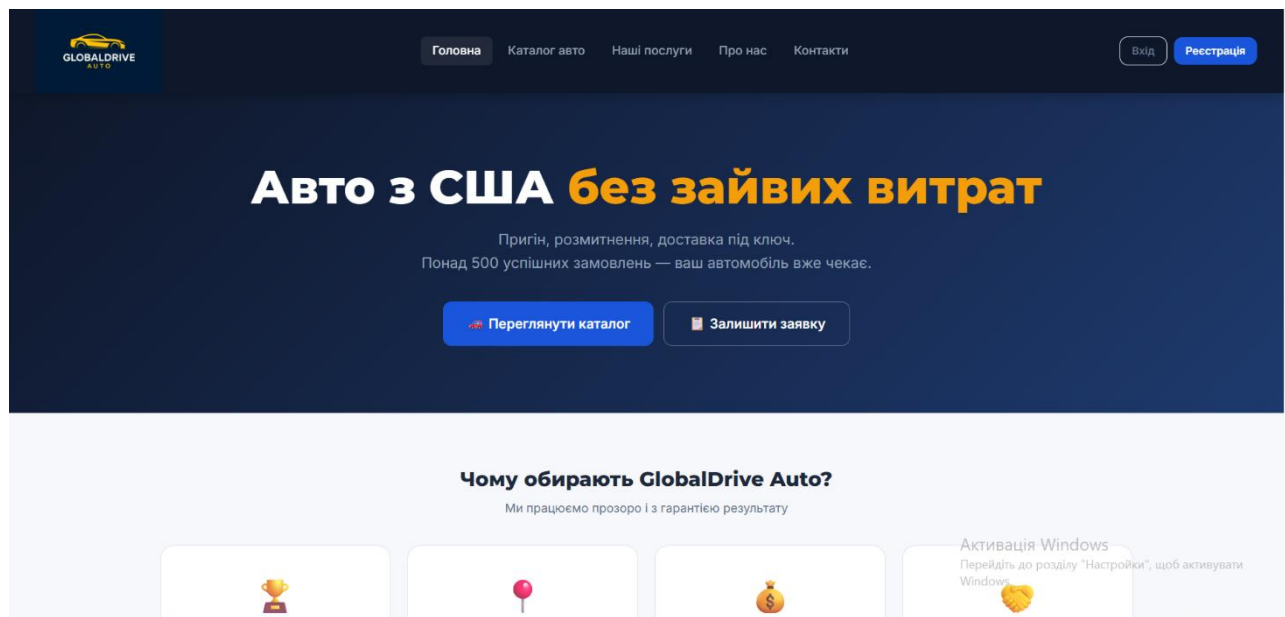
### **3.2. Макети сторінок веб-сайту**

На основі розробленої карти сайту було спроектовано та реалізовано макети всіх сторінок веб-додатку автосалону. При проектуванні дизайну особлива увага приділялась таким аспектам: чіткість і простота навігації, інтуїтивна зрозумілість інтерфейсу, адаптивність до різних розмірів екрана, єдність візуального стилю всіх сторінок.

Кольорова гама сайту автосалону «GlobalDrive Auto» побудована на поєднанні темно-синього (як основного фірмового кольору), білого та сірого з акцентами золотистого. Темно-синій колір асоціюється з надійністю, преміальністю та професіоналізмом, що відповідає позиціонуванню автосалону як надійного партнера у питанні пригону автомобілів. Білий і світло-сірий тони використовуються для основного фону сторінок та забезпечують гарну читабельність текстового контенту. Золотисті акценти у логотипі та інтерактивних елементах (кнопках, посиланнях при наведенні) додають дизайну елегантності й підкреслюють преміальний характер послуг.

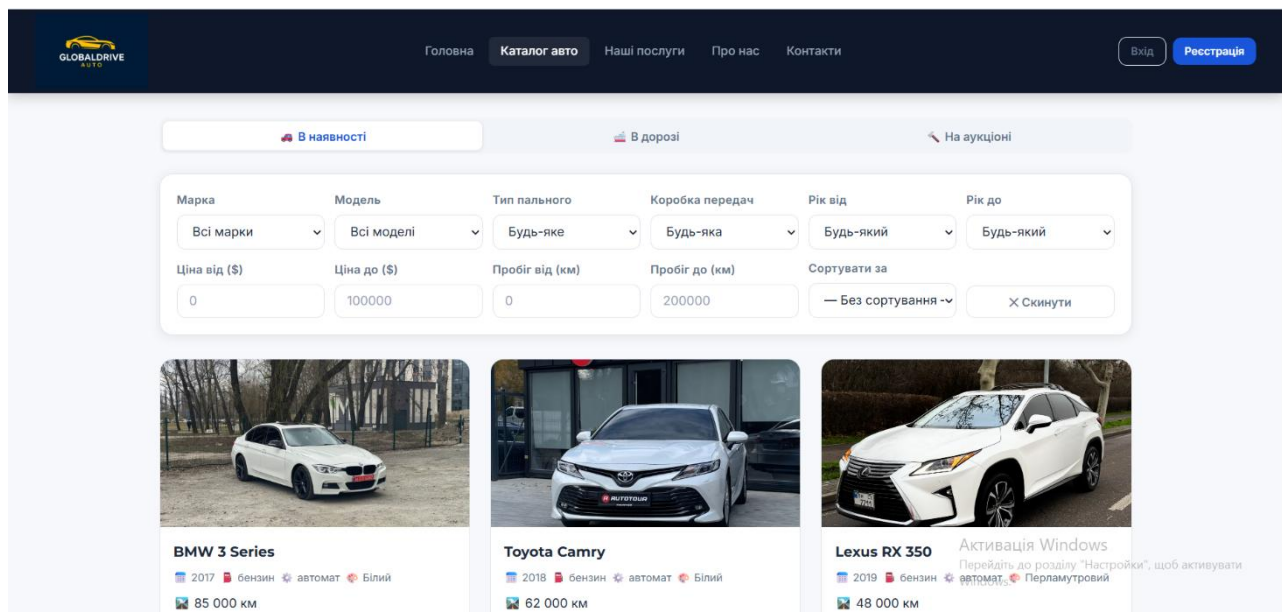
Усі сторінки сайту мають єдину структуру, що складається з трьох основних блоків: верхньої навігаційної панелі (навбару) з логотипом автосалону та посиланнями на основні розділи, центральної контентної частини, специфічної для кожної сторінки, та підвалу (футера) з контактною інформацією, посиланнями на соціальні мережі та копірайтом. Висота навбару становить 100 пікселів, що дозволяє розмістити повноформатний логотип і забезпечує впізнаваність бренду.

Головна сторінка (index.html) відкриває користувачеві презентаційний банер з фоновим зображенням преміального автомобіля, основним слоганом автосалону та кнопкою переходу до каталогу. Далі розташовано блок «Наші переваги» з виокремленням ключових цінностей для клієнта (надійність, прозорість, повний супровід, гарантія якості), блок з прикладами найкращих пропозицій каталогу та форма швидкого зв'язку з менеджером (рис. 3.2).



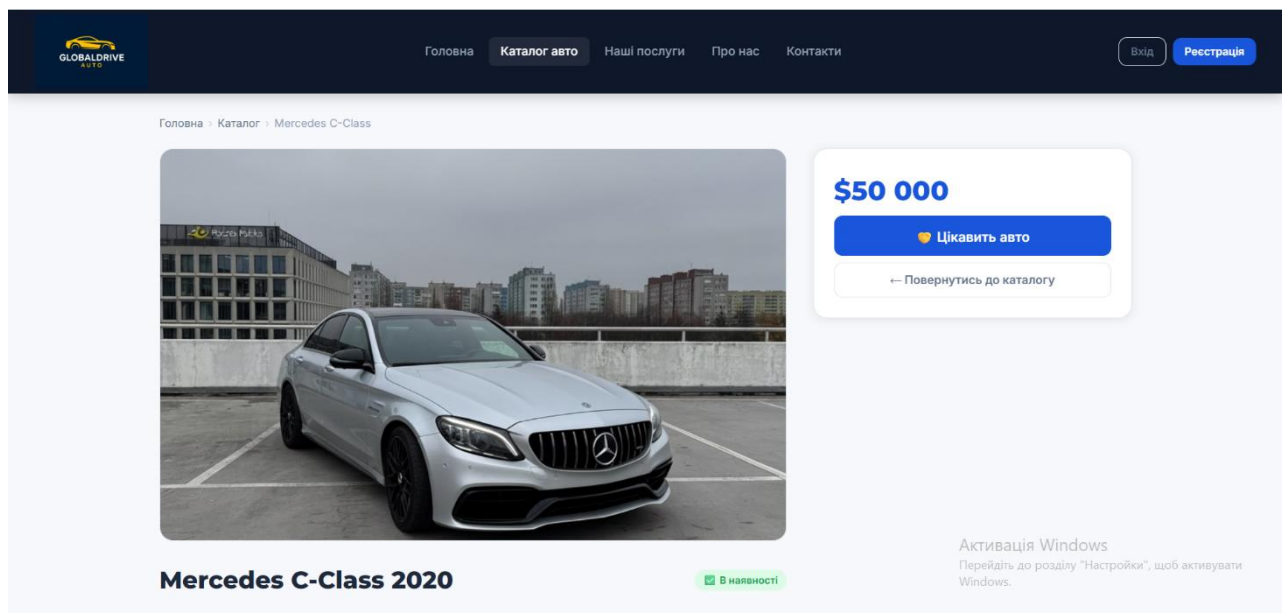
**Рис. 3.2. Головна сторінка сайту**

Сторінка каталогу (catalog.html) є однією з ключових для клієнтів. У верхній частині розміщено панель фільтрів, що дозволяє звузити пошук за маркою, моделлю, роком випуску, типом палива, типом коробки передач та ціновим діапазоном. У центральній частині відображаються картки автомобілів з фотографією, основними характеристиками та ціною. (рис. 3.3).

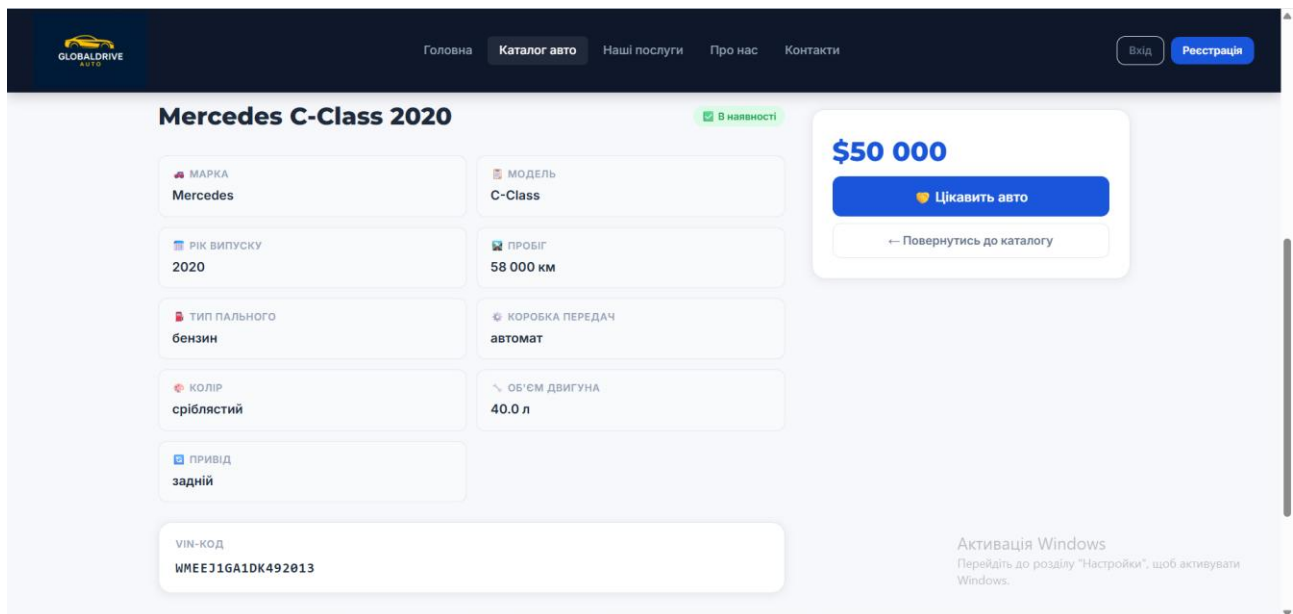


**Рис. 3.3. Каталог автомобілів**

Детальна сторінка автомобіля (car.html) містить повний опис обраного авто: велике зображення, повну специфікацію (марка, модель, рік випуску, об'єм двигуна, тип палива, пробіг, колір, тип трансмісії, тип приводу). (рис. 3.4- рис. 3.5).

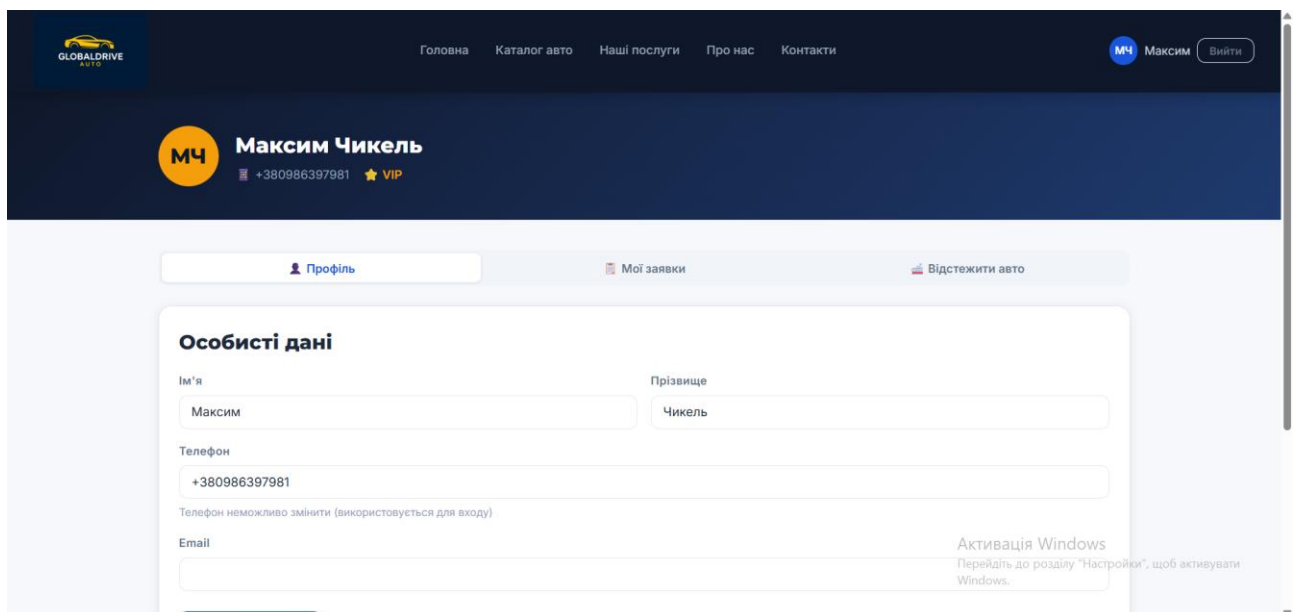


**Рис. 3.4. Сторінка авто з детальними характеристиками**



**Рис. 3.5. Сторінка авто з детальними характеристиками**

Особистий кабінет клієнта (cabinet.html) реалізовано у вигляді односторанкового додатку з трьома вкладками. На вкладці «Мої заявки» відображається список усіх заявок клієнта з їхніми поточними статусами та датами подання. На вкладці «Відстежити авто» представлено укладені договори з інформацією про автомобіль, суму та статус виконання, також клієнт бачить хронологію переміщення автомобіля від моменту купівлі на аукціоні до доставки в Україну (рис. 3.6, рис.3.7, рис. 3,8).



**Рис. 3.6. Особистий кабінет клієнта**

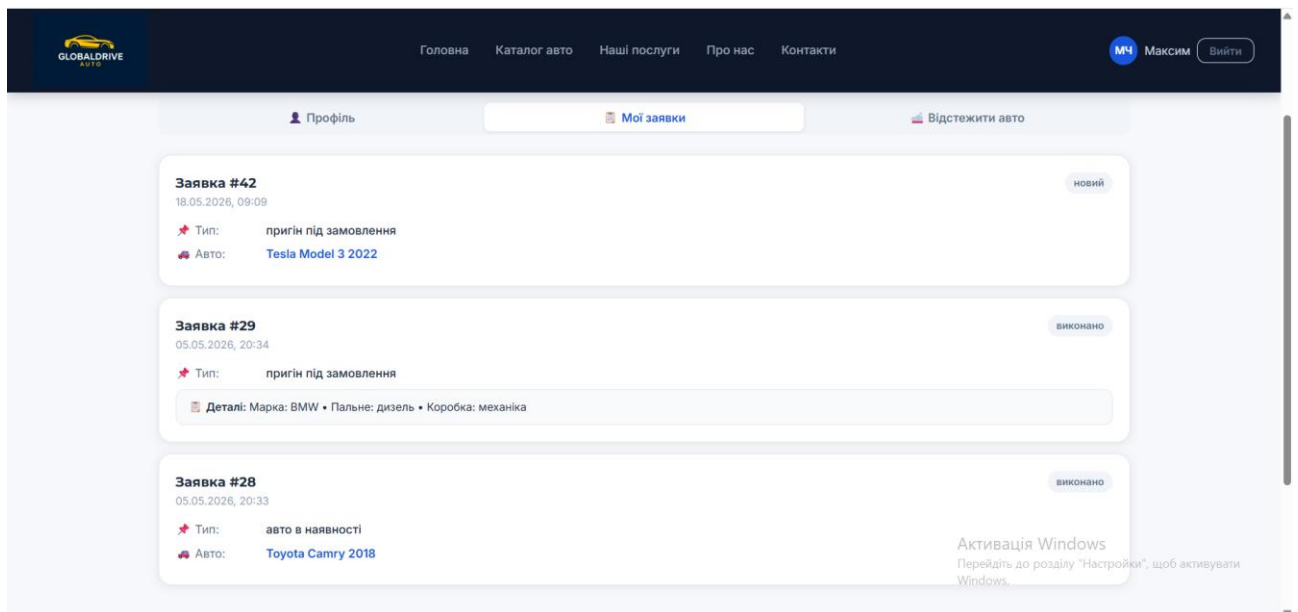


Рис.3.7 Заявки клієнта

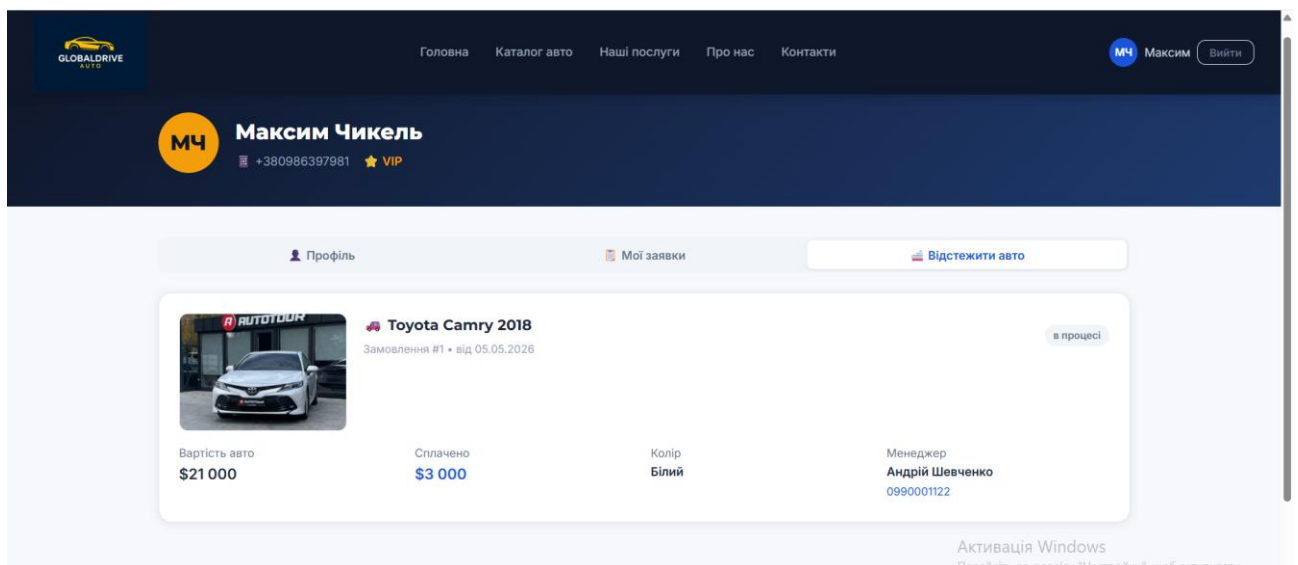
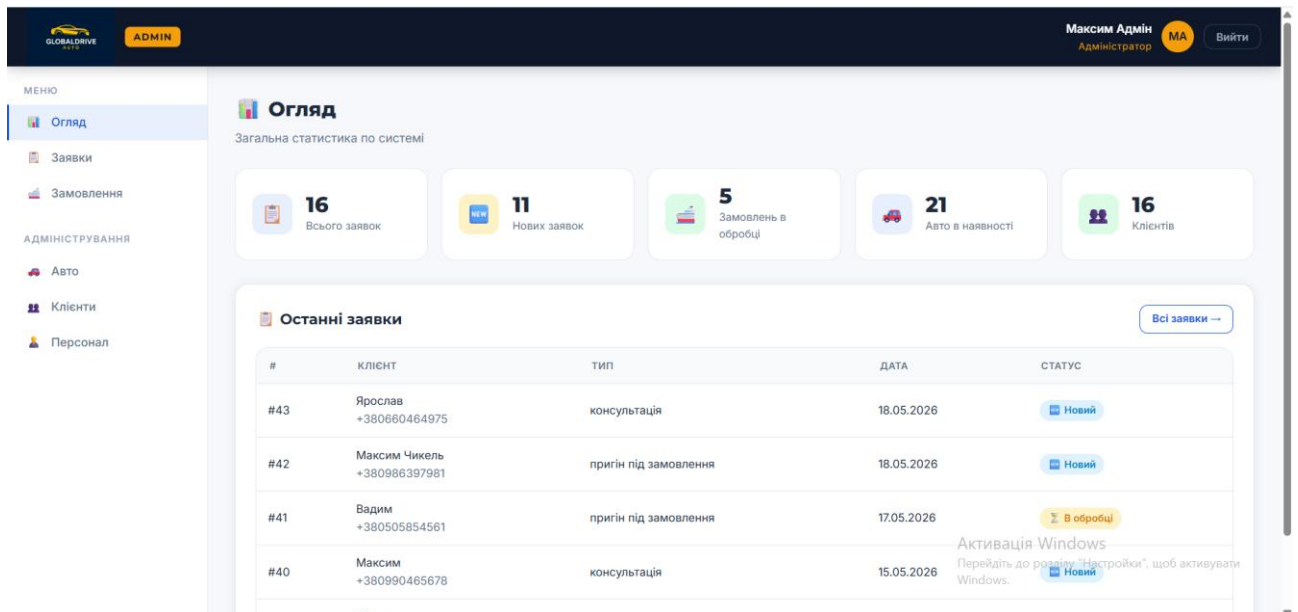


Рис.3.8 Вкладка відстежити авто

Адміністративна панель (admin.html) виконана у класичному стилі desktop-додатку з боковою навігацією зліва та робочою областю праворуч. У боковому

меню розташовано пункти: «Дашборд» (з агрегованою статистикою), «Заявки», «Замовлення», «Каталог авто», «Клієнти», «Персонал» (лише для адміністратора). У верхній частині відображається інформація про авторизованого працівника та кнопка виходу з системи. Робоча область містить таблиці записів з можливістю фільтрації, пошуку, редагування та видалення (рис. 3.9).



**Рис. 3.9. Адміністративна панель**

Усі сторінки сайту реалізовано з урахуванням принципів адаптивного дизайну: при перегляді на пристроях з різною роздільною здатністю верстка автоматично перебудовується для забезпечення зручного користувацького досвіду. На мобільних пристроях навігаційне меню згортається у бургер-кнопку, а блоки контенту перебудовуються у вертикальний стек.

### 3.3. Програмування серверної частини

Серверна частина інформаційної системи автосалону реалізована на платформі Node.js з використанням фреймворку Express.js. Архітектура серверного додатку побудована за принципом розподілу відповідальностей: основний файл `server.js` відповідає за ініціалізацію Express-застосунку, підключення `middleware` та реєстрацію маршрутів, тоді як обробку конкретних запитів вилучено в окремі модулі, що знаходяться в каталозі `routes/`. Така

структура полегшує супровід коду та забезпечує масштабованість додатку. [6, 9, 10]

Основними серверними модулями інформаційної системи є:

`server.js` - основний файл, що ініціалізує Express-застосунок, підключає `middleware`-функції (`cors`, `express.json`, `express.static`), реєструє всі маршрути API, встановлює з'єднання з базою даних через пул підключень `mysql2` і запускає HTTP-сервер на заданому порту;

`middleware/auth.js` - реалізує систему перевірки JWT-токенів та контролю доступу за ролями. Містить чотири основні функції: `requireClient` (доступ лише для авторизованих клієнтів), `requireEmployee` (доступ для будь-якого працівника - менеджера або адміністратора), `requireAdmin` (доступ виключно для адміністратора) та `requireAnyAuth` (доступ для будь-якого авторизованого користувача незалежно від ролі). Ці функції підключаються до конкретних маршрутів API і виконуються перед основним обробником запиту, забезпечуючи централізований контроль безпеки;

`routes/auth.js` - реалізує реєстрацію нових клієнтів, авторизацію клієнтів, перевірку JWT-токена та отримання інформації про поточного авторизованого користувача;

`routes/admin-auth.js` - окремий модуль авторизації для працівників автосалону (менеджерів та адміністраторів). Виокремлення цієї логіки в окремий файл відповідає принципу розподілу відповідальностей і дозволяє використовувати різні правила валідації та різний формат JWT-токенів для клієнтів і працівників;

`routes/cars.js` - забезпечує CRUD-операції з каталогом автомобілів: отримання списку з фільтрацією за маркою, моделлю, ціною та іншими параметрами, отримання детальної інформації за `car_id`, додавання нового автомобіля, редагування та видалення записів;

`routes/requests.js` - обробляє подання нових заявок клієнтами, отримання списку заявок (з фільтрацією за статусом і менеджером), зміну статусу заявки, призначення менеджера на заявку;

`routes/orders.js` - реалізує оформлення замовлень на основі заявок, отримання списку замовлень, зміну статусу оплати та виконання, повне редагування замовлення;

`routes/tracking.js` - забезпечує ведення журналу логістики: додавання нових подій доставки менеджером, видалення записів, отримання повної історії переміщення авто для клієнта в особистому кабінеті та для менеджера в адміністративній панелі;

`routes/clients.js` та `routes/employees.js` - реалізують керування клієнтською базою та персоналом для адміністратора (перегляд, редагування, видалення записів);

`routes/upload.js` - обробляє завантаження фотографій автомобілів. Для роботи з файлами використовується npm-бібліотека `multer`, яка забезпечує приймання файлів у форматі `multipart/form-data`, валідацію розміру (максимум 5 МБ) та типу файлу (дозволені лише `.jpg`, `.jpeg`, `.png`, `.webp`). Завантажені фотографії зберігаються у каталозі `public/uploads/cars/` із автоматичним перейменуванням за шаблоном `car_{id}_{timestamp}.{ext}` для уникнення колізій імен.

Особливістю реалізації серверної логіки заявок є автоматичний розподіл нових заявок між менеджерами за принципом їхньої спеціалізації. Цей розподіл здійснюється безпосередньо на серверній стороні при подачі заявки клієнтом, без необхідності ручного втручання адміністратора. Якщо клієнт обирає тип заявки, що стосується наявного автомобіля (наприклад, з картки конкретного авто на сторінці каталогу), система автоматично призначає її менеджеру з посадою «Менеджер авто в наявності»; якщо ж клієнт подає заявку на пригін авто на замовлення, заявка направляється менеджеру з посадою «Менеджер з пригону»; нарешті, заявки типу «консультація» автоматично передаються адміністратору, який або консультує клієнта самотійно, або переадресовує запит профільному менеджеру через адміністративну панель. Технічно цей механізм реалізується через SQL-запит до таблиці `Employees` з фільтрацією за полем `position`, що дозволяє знайти першого вільного працівника відповідної спеціалізації. Така



архітектурна особливість дозволяє автосалону розділити робочі процеси між профільними фахівцями та значно прискорити обробку клієнтських запитів.

Додатковою важливою деталлю серверної реалізації подання заявки є автоматична нормалізація телефонного номера клієнта. Перш ніж зберегти заявку в базі даних, серверний код виконує приведення номера до єдиного канонічного формату `+380XXXXXXXXXX`: видаляються пробіли та дефіси, локальні номери у форматі `0XXXXXXXXXX` доповнюються кодом країни `+38`, а номери, які вже мають префікс `380`, але без знаку `+`, отримують його. Така нормалізація є необхідною умовою для коректної роботи зв'язку заявок із обліковими записами клієнтів через поле `client_phone`, описаного у розділі 2.2.

Реалізація `middleware`-функцій у файлі `middleware/auth.js` є наочним прикладом застосування патерну «`middleware`-ланцюг» у фреймворку `Express.js`. Коли `HTTP`-запит надходить на захищений маршрут, він спочатку проходить через відповідну `middleware`-функцію, яка перевіряє наявність та валідність `JWT`-токена у заголовку `Authorization`. У разі успішної перевірки управління передається основному обробнику запиту через виклик функції `next()`, у разі помилки - клієнту повертається відповідь зі статусом `401 Unauthorized` або `403 Forbidden`. Такий підхід дозволяє винести логіку безпеки в окремий компонент, який легко тестувати, модифікувати та повторно використовувати для різних маршрутів.

Підключення до бази даних реалізовано через бібліотеку `mysql2` з використанням пулу з'єднань (`connection pool`). Такий підхід суттєво підвищує продуктивність серверної частини за рахунок повторного використання вже встановлених з'єднань замість створення нових для кожного запиту. Параметри підключення (адреса сервера, порт, ім'я користувача, пароль, назва бази даних) винесено в окремий конфігураційний файл, що відповідає сучасним практикам розробки і дозволяє легко змінювати налаштування при розгортанні системи в різних середовищах.

Для безпеки облікових даних користувачів усі паролі зберігаються у базі не у відкритому вигляді, а у вигляді хешів, обчислених за алгоритмом `bcrypt` з

параметром `salt rounds = 10`. Перевірка пароля при авторизації відбувається через функцію `bcrypt.compare`, яка порівнює введений користувачем пароль з хешем у базі. Така схема забезпечує надійний захист навіть у випадку несанкціонованого доступу до бази даних: відновити вихідні паролі з хешів практично неможливо. [6]

Автентифікація користувачів реалізована через JSON Web Tokens (JWT). При успішному вході сервер генерує токен, у який задовано ідентифікатор користувача, його роль (клієнт / менеджер / адміністратор) та термін дії токена. Токен повертається клієнтській частині й зберігається в браузері (у `localStorage`). При кожному наступному запиті до захищених маршрутів клієнт передає токен у HTTP-заголовку `Authorization`. Сервер перевіряє валідність токена через `middleware` та визначає права доступу. Розглянемо приклад реалізації серверного маршруту авторизації (лістинг 3.1). `route/auth.js`

### *Лістинг 3.1. Серверна реалізація авторизації користувача*

```
// Вхід клієнта
router.post('/login', (req, res) => {
  let { phone, password } = req.body;

  if (!phone || !password) {
    return res.status(400).json({ error: 'Введіть телефон і пароль!' });
  }

  // Нормалізуємо телефон
  phone = normalizePhone(phone);

  db.query('SELECT * FROM Clients WHERE phone = ?', [phone], async
(err, results) => {
    if (err) return res.status(500).json({ error: err.message });
    if (results.length === 0) return res.status(400).json({ error:
'Клієнта не знайдено!' });

    const client = results[0];

    // Перевіряємо пароль
    const isMatch = await bcrypt.compare(password,
client.password);
    if (!isMatch) return res.status(400).json({ error: 'Невірний
пароль!' });

    // Генеруємо токен
    const token = jwt.sign(
      { client_id: client.client_id, phone: client.phone },
      JWT_SECRET,
```

```

        { expiresIn: '7d' }
    );

    res.json({
      message: 'Вхід успішний!',
      token,
      client: {
        client_id: client.client_id,
        first_name: client.first_name,
        last_name: client.last_name,
        phone: client.phone
      }
    });
  });
});

```

Наведений код виконує пошук користувача у таблиці Clients за нормалізованим телефоном, перевіряє хеш пароля через bcrypt.compare і, у разі успіху, генерує JWT-токен з ідентифікатором клієнта та його телефоном. Токен має термін дії 7 днів, після чого користувач повинен повторно увійти у систему. Окремий маршрут авторизації для працівників автосалону (адміністратора та менеджерів) реалізовано у файлі routes/admin-auth.js і відрізняється тим, що логін здійснюється за email-адресою, а в JWT-токен додатково записується посада працівника.

Іншим важливим маршрутом є подання нової заявки клієнтом. Цей маршрут реалізує бізнес-логіку перевірки коректності даних, збереження заявки у базі та повернення клієнту повідомлення про успіх (лістинг 3.2). routes/requests

### *Лістинг 3.2. Подання нової заявки клієнтом*

```

router.post('/', (req, res) => {
  let { client_name, client_phone, request_type,
        car_id, search_context } = req.body;

  // Нормалізація телефону - щоб співпадав з форматом клієнтів
  if (client_phone) {
    client_phone = String(client_phone).replace(/\s/g,
    '').replace(/-/g, '');
    if (client_phone.startsWith('0'))
      client_phone = '+38' + client_phone;
    if (!client_phone.startsWith('+') &&
    client_phone.startsWith('380'))
      client_phone = '+' + client_phone;
  }

  // Визначення посади менеджера за типом заявки
  let position = 'Менеджер з пригону';
  if (request_type && request_type.includes('наявн')) {
    position = 'Менеджер авто в наявності';
  }

```

```

    }
    if (request_type === 'консультація') {
        position = 'Адміністратор';
    }

    // Пошук відповідного працівника та збереження заявки
    db.query(
        'SELECT employee_id FROM Employees WHERE position = ? LIMIT
1',
        [position],
        (err, employees) => {
            if (err) return res.status(500).json({ error: err.message
});

            const employee_id = employees.length > 0
                ? employees[0].employee_id : null;
            db.query(
                `INSERT INTO Requests
                (client_name, client_phone, request_type, car_id,
                search_context, employee_id)
                VALUES (?, ?, ?, ?, ?, ?)`,
                [client_name, client_phone, request_type,
                car_id || null,
                search_context ? JSON.stringify(search_context) :
null,
                employee_id],
                (err, result) => {
                    if (err) return res.status(500).json({ error:
err.message });

                    res.json({ message: 'Заявку створено!',
                        request_id: result.insertId });
                }
            );
        }
    );
});

```

Розглянемо приклад реалізації middleware-функції для перевірки прав адміністратора (лістинг 3.3).

### Лістинг 3.3. Middleware-функція для перевірки прав адміністратора Javascript

```

// Дістати і перевірити JWT-токен з заголовка Authorization
function verifyToken(req) {
    const token = req.headers.authorization?.split(' ')[1];
    if (!token) return null;
    try {
        return jwt.verify(token, JWT_SECRET);
    } catch (err) {
        return null;
    }
}

// Тільки адміністратор
function requireAdmin(req, res, next) {
    const decoded = verifyToken(req);
    if (!decoded || !decoded.employee_id) {

```

```

        return res.status(401).json({ error: 'Потрібна авторизація'
    });
    }
    if (decoded.position !== 'Адміністратор') {
        return res.status(403)
            .json({ error: 'Доступ заборонено: тільки для
адміністратора' });
    }
    req.employee = decoded;
    next();
}

module.exports = { requireEmployee, requireAdmin,
                    requireClient, requireAnyAuth };

```

Функція виконує дві послідовні перевірки через допоміжну функцію `verifyToken`: спочатку перевіряється наявність та криптографічна валідність JWT-токена (через `jwt.verify`), а потім — наявність ідентифікатора працівника у токені та відповідність ролі. Тільки якщо всі перевірки пройшли успішно, управління передається наступному обробнику через виклик `next()`. Подібним чином реалізовані інші `middleware`-функції — `requireEmployee` (доступ для будь-якого працівника), `requireClient` (доступ лише для клієнтів) та `requireAnyAuth` (доступ для будь-якого авторизованого користувача).

### 3.4. Програмування клієнтської частини

Клієнтська частина інформаційної системи реалізована з використанням стандартних веб-технологій: HTML5 для семантичної розмітки сторінок, CSS3 для оформлення зовнішнього вигляду та забезпечення адаптивності, JavaScript для додавання інтерактивності та асинхронної взаємодії з сервером через REST API.

Структура клієнтської частини включає такі основні JavaScript-файли:

- `js/auth.js` -модуль авторизації, який обробляє форми входу та реєстрації, зберігає JWT-токен у `localStorage` браузера, виконує редирект на відповідну сторінку після успішного входу та забезпечує вихід з системи;
- `js/catalog.js` -модуль каталогу: завантажує список автомобілів з сервера, рендерить картки, реалізує клієнтську фільтрацію та сортування;

-js/car.js -обробляє детальну сторінку автомобіля, у тому числі форму подання заявки;

-js/cabinet.js -реалізує особистий кабінет клієнта: завантажує заявки, замовлення та інформацію про доставку;

-js/admin.js -найбільший модуль, що відповідає за всі функції адміністративної панелі: відображення статистики, керування заявками, замовленнями, каталогом, клієнтами та персоналом.

Взаємодія клієнтської частини з сервером відбувається через Fetch API - сучасний інтерфейс для виконання асинхронних HTTP-запитів [7, 8]. Усі запити до захищених маршрутів обов'язково містять JWT-токен у заголовку Authorization. Розглянемо приклад функції завантаження списку автомобілів з фільтрами (лістинг 3.4). js/catalog.js

#### *Лістинг 3.4. Завантаження каталогу авто з фільтрацією*

```
let allCars = [];  
let currentTab = 'available';  
  
async function loadCars() {  
  showLoading();  
  hideBanner();  
  try {  
    let url = '';  
    if (currentTab === 'available') url = `${API}/cars`;  
    if (currentTab === 'transit')    url = `${API}/cars/in-  
transit`;  
    if (currentTab === 'auction')   url = `${API}/cars/auction`;  
  
    const res = await fetch(url);  
    allCars = await res.json();  
    populateFilters();  
    renderCars(allCars);  
  } catch (err) {  
    hideLoading();  
    showEmpty();  
  }  
}
```

Функція loadCars працює у трьох режимах залежно від активної вкладки на сторінці каталогу: «Авто в наявності», «Авто в дорозі» та «Авто на аукціоні». Серверна частина повертає JSON-масив автомобілів з відповідним статусом, після чого функція populateFilters заповнює випадні списки доступних значень фільтрів (марки, моделі, типу палива), а функція renderCars

динамічно формує HTML-картки автомобілів і додає їх до контейнера каталогу. Завдяки використанню шаблонних рядків (template literals) JavaScript та асинхронної моделі Fetch API код виглядає лаконічно та легко читається.

Для подання заявки з картки автомобіля реалізована функція `submitInterest`, що збирає дані з модального вікна, перевіряє їх на валідність, виконує клієнтську нормалізацію телефону та надсилає запит на сервер. Особливістю функції є **динамічне визначення типу заявки** на основі поточного статусу автомобіля: якщо авто реально стоїть на майданчику автосалону (статус «в наявності»), заявка автоматично отримує тип «авто в наявності» і потрапляє до менеджера авто в наявності; для всіх інших статусів («в дорозі», «на аукціоні») заявка стає «пригін під замовлення» і направляється менеджеру з пригону. Така архітектура працює в тандемі з логікою серверного роутингу, описаною у лістингу 3.2 (лістинг 3.5). `js/car.js`

### Лістинг 3.5. Подання заявки з картки автомобіля

*Лістинг 3.5. Подання заявки з картки автомобіля*

```
/* — ВІДПРАВИТИ ЗАЯВКУ — */
async function submitInterest() {
  const name      = document.getElementById('req-name').value.trim();
  let  phone      = document.getElementById('req-phone').value.trim();
  const comment   = document.getElementById('req-comment')?.value.trim() || '';
  const alertEl   = document.getElementById('modal-alert');
  const btn       = document.querySelector('#interest-modal .btn-primary');

  alertEl.style.display = 'none';
  btn.disabled = true;
  btn.textContent = 'Надсилаємо...';

  if (!name || !phone) {
    alertEl.className = 'alert alert-error mb-16';
    alertEl.innerHTML = "❌ Введіть ім'я та телефон!";
    alertEl.style.display = 'flex';
    alertEl.scrollIntoView({ behavior: 'smooth', block: 'center' });
    btn.disabled = false;
    btn.textContent = 'Надіслати заявку';
    return;
  }

  phone = phone.replace(/\s/g, '').replace(/-/g, '');
  if (phone.startsWith('0')) phone = '+38' + phone;
```

```

// ⚡ ДОДАЄМО ДИНАМІЧНУ ЛОГІКУ ТИПУ ЗАЯВКИ
// Якщо авто реально стоїть на майданчику:
let dynamicRequestType = 'пригін під замовлення';

if (currentCar && currentCar.car_status === 'в наявності') {
    dynamicRequestType = 'авто в наявності';
}

// Тепер, якщо статус 'в дорозі' або 'на аукціоні', змінна
залишиться 'пригін під замовлення'

try {
    const res = await fetch(`${API}/requests`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
            client_name: name,
            client_phone: phone,
            request_type: dynamicRequestType, // ← Відправляємо розумну
змінну
            car_id: currentCar.car_id,
            search_context: comment ? { comment } : null
        })
    });

    const data = await res.json();

    if (!res.ok) {
        alertEl.className = 'alert alert-error mb-16';
        alertEl.innerHTML = `❌ ${data.error || 'Помилка'}`;
        alertEl.style.display = 'flex';
        btn.disabled = false;
        btn.textContent = 'Надіслати заявку';
        return;
    }

    alertEl.className = 'alert alert-success mb-16';
    alertEl.innerHTML = "✅ Заявку надіслано! Менеджер зв'яжеться з
вами найближчим часом.";
    alertEl.style.display = 'flex';

```

Адаптивний дизайн сайту забезпечується за допомогою медіа-запитів CSS3 та flex/grid-розкладок. Для основних точок переходу (768 пікселів - планшети, 480 пікселів - мобільні пристрої) визначено окремі правила стилізації, які перебудовують верстку для оптимального відображення. Зокрема, на мобільних пристроях навігаційне меню згортається у бургер-кнопку, фільтри каталогу переміщуються у згортний блок, а картки автомобілів розташовуються в одну колонку.

Для покращення користувацького досвіду на сторінках реалізовано низку додаткових інтерактивних можливостей: модальні вікна для підтвердження дій (наприклад, видалення запису в адмін-панелі), впливаючі повідомлення (toast



notifications) для індикації успіху або помилки виконання операцій, динамічне завантаження вмісту без перезавантаження сторінки.

### **3.5. Розміщення веб-сайту на локальному середовищі**

Для демонстрації роботи інформаційної системи автосалону «GlobalDrive Auto» в умовах, наближених до реального продакшн-середовища, розгортання здійснено на локальному віртуальному середовищі. Як гіпервізор обрано VMware Workstation Pro, що дозволяє створювати та керувати віртуальними машинами на базі робочої станції розробника. На віртуальну машину встановлено серверну версію операційної системи Ubuntu Server LTS - стабільний і широко використовуваний дистрибутив Linux, оптимізований для серверних застосувань.

Конфігурація віртуальної машини включає такі параметри: 2 виділених процесорних ядра, 4 ГБ оперативної пам'яті, 25 ГБ дискового простору, мережевий адаптер у режимі NAT з фіксованою IP-адресою 192.168.64.133. Така конфігурація є достатньою для роботи серверної частини додатку, бази даних та обробки тестового навантаження.

Процес підготовки серверного середовища складався з кількох послідовних етапів:

- встановлення Node.js та менеджера пакетів npm з офіційного репозиторію NodeSource у версії 20.x LTS;
- встановлення MySQL Server, виконання базового налаштування безпеки та створення користувача бази даних з відповідними правами;
- імпорт SQL-скрипту створення таблиць у MySQL Server та наповнення бази тестовими даними;
- завантаження вихідного коду проекту на віртуальну машину та встановлення npm-залежностей командою `npm install`;
- конфігурація змінних середовища (адреса бази даних, ім'я користувача, пароль, секретний ключ JWT) у файлі `.env`;

-налаштування міжмережевого екрану `ufw` для дозволу вхідних з'єднань на порту 3000 (порт серверної частини).

Запуск серверної частини виконується командою `node server.js` з кореневого каталогу проекту. Сервер починає прослуховування на TCP-порту 3000 і обробляє вхідні HTTP-запити. Доступ до сайту з хост-машини здійснюється за адресою `http://192.168.64.133:3000/`, що дозволяє тестувати веб-додаток у звичайному браузері.

Розробка та налагодження коду виконується у середовищі Visual Studio Code, що підключене до віртуальної машини через протокол SSH за допомогою розширення Remote -SSH. Такий підхід забезпечує зручність редагування коду безпосередньо на цільовій машині без необхідності постійно копіювати файли між хостом і віртуальною машиною. Будь-які зміни у коді відразу ж застосовуються при перезапуску сервера.

Розміщення інформаційної системи на локальному віртуальному середовищі дозволяє повністю емулювати умови продакшн-сервера, тестувати додаток у контрольованому ізолюваному середовищі та проводити демонстрацію функціональності без необхідності оренди реального серверного хостингу. При переході системи у промислову експлуатацію розгортання може бути виконане на віртуальному виділеному сервері (VPS) у будь-якого хостинг-провайдера без принципових змін в архітектурі додатку.

## ВИСНОВКИ

У ході виконання курсової роботи було досліджено особливості автоматизації бізнес-процесів автосалону, що спеціалізується на пригоні автомобілів з-за кордону, та розроблено інформаційну систему, яка комплексно вирішує поставлені завдання. Здобуті результати дозволяють зробити такі основні висновки.

У першому розділі було проведено детальний аналіз предметної області. Виявлено основні бізнес-процеси автосалону: ведення каталогу автомобілів, прийом клієнтських заявок на пригін чи консультацію, оформлення замовлень (договорів), відстеження логістики доставки та підтримання клієнтської бази. Розроблено модель варіантів використання, у якій виділено трьох акторів системи -клієнта, менеджера та адміністратора, кожен із яких має власний набір функціональних можливостей. На основі техніко-економічного обґрунтування обрано оптимальний технологічний стек: Node.js з фреймворком Express.js для серверної частини, HTML5/CSS3/JavaScript для клієнтської частини та СКБД MySQL для зберігання даних.

У другому розділі було спроектовано реляційну базу даних, що складається з шести взаємопов'язаних таблиць: Cars, Clients, Employees, Requests, Orders та Delivery\_Tracking. Виконано перевірку схеми бази даних на відповідність першим трьом нормальним формам, обґрунтовано свідому денормалізацію окремих полів задля можливості прийому заявок від незареєстрованих відвідувачів. Визначено типи даних для кожного атрибута з урахуванням характеру інформації, що зберігається. Реалізовано механізми забезпечення цілісності даних через первинні та зовнішні ключі, обмеження UNIQUE на унікальні поля (email, VIN-код), обмеження типу ENUM на поля з фіксованим переліком значень. Розроблено SQL-скрипт створення таблиць та реалізовано низку запитів, що задовольняють основні аналітичні та операційні потреби системи.

У третьому розділі реалізовано повнофункціональний веб-додаток. Розроблено клієнтську частину з публічними сторінками (головна, каталог, картка авто, послуги, про нас, контакти), сторінками авторизації та реєстрації, особистим кабінетом клієнта та адміністративною панеллю. На серверній частині реалізовано модулі обробки запитів за принципом REST API: модуль автентифікації (з використанням bcrypt для хешування паролів та JWT-токенів для контролю сесій), модулі CRUD-операцій з основними сутностями системи, модуль обробки заявок та модуль ведення журналу логістики. Веб-додаток розгорнуто на локальному віртуальному середовищі VMware Workstation з операційною системою Ubuntu Server, що дозволяє повністю емулювати умови продакшн-сервера.

Розроблена інформаційна система забезпечує автоматизацію всіх ключових бізнес-процесів автосалону, мінімізує ризик помилок при ручному веденні обліку, надає клієнтам зручний онлайн-доступ до інформації про автомобілі та статус їхніх замовлень, а керівництву - інструменти аналітичної звітності для прийняття обґрунтованих управлінських рішень. Завдяки чіткому розмежуванню прав доступу між клієнтами, менеджерами та адміністраторами забезпечується захист чутливої інформації та коректний розподіл функціональних обов'язків персоналу.

## СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Пасічник В. В. Організація баз даних та знань / В. В. Пасічник, В. А. Резніченко. -К. : Видавнича група BHV, 2006. -384 с.
2. Берко А. Ю. Системи баз даних та знань. Книга 1. Організація баз даних та знань : навчальний посібник / А. Ю. Берко, О. М. Верес, В. В. Пасічник. - 2-ге вид. -Львів : Магнолія 2006, 2015. -440 с.
3. Балик Н. Бази даних MySQL : навчальний посібник / Н. Балик, В. Мандзюк. -Тернопіль : Навчальна книга -Богдан, 2010. -160 с.
4. Двірничук К. В. Веб-програмування та веб-дизайн : навчальний посібник / К. В. Двірничук, Д. О. Вацек. -Чернівці : Чернівецький національний університет ім. Ю. Федьковича, 2022. -472 с.
5. Баран С. В. Основи web-програмування : навчальний посібник / С. В. Баран. -Кривий Ріг : Державний університет економіки і технологій, 2023. -316 с.
6. Brown E. Web Development with Node and Express: Leveraging the JavaScript Stack / E. Brown. 2nd ed. -Sebastopol : O'Reilly Media, 2019. -346 p.
7. MDN Web Docs. JavaScript [Електронний ресурс] / Mozilla Developer Network. - <https://developer.mozilla.org/uk/docs/Web/JavaScript>
8. Сучасний підручник з JavaScript [Електронний ресурс]. - <https://uk.javascript.info/>

9. Node.js Documentation [Электронный ресурс] / OpenJS Foundation. -  
<https://nodejs.org/docs/latest/api/>

10. Express.js Routing Guide [Электронный ресурс] / OpenJS Foundation. -  
<https://expressjs.com/en/guide/routing.html>

11. MySQL 8.0 Reference Manual [Электронный ресурс] / Oracle  
Corporation. - <https://dev.mysql.com/doc/refman/8.0/en/>

12. JSON Web Tokens - Introduction [Электронный ресурс] / Auth0 by Okta. -  
<https://jwt.io/introduction>